

Dynamic Programming Basics

09/13/2026 09:49:21

<https://chatgpt.com/c/6aa52dee-8fec-83ee-b412-df8f6325fec8>

dp-pattern-cheat-sheet.pdf go through it completely , i want to learn , t...

Thinking ● Identified DP definitions and classifications ✓ Worked for 8s...

we'll go pattern by pattern starting with prefix , explain it , give a...

Yes. We'll go pattern by pattern, and for each one I'll build the pattern...

let's solve some problem on this , just state the problem , i'll try to find...

Problem 1 — Climbing Stairs You are climbing a staircase with n steps....

1. ways to climb upto the stair i 2. jump 3. i - 1 , i - 2 4. $dp[i] = dp[i - 1] + dp[i - 2]$...

Exactly right. One small wording improvement: State: $dp[i]$ = number o...

1. maximum amount of money stolen considering upto house i 2. rob it ...

Perfect. Your recurrence is correct. Only one detail to add: for the...

1. ways to decode string $s[0...i]$ 2. splitting the string 3. $dp[i - 1]$, $dp[i - 2]$...

Yes — your recurrence idea is correct, and you're starting to see the...

1. min coins to amount to amount i 2. adding a coin to the previous...

Your Coin Change derivation is almost correct, but there is one missing...

next

Problem 5 — Longest Increasing Subsequence (LIS) Given an array num...

1. LIS ending at $nums[0...i]$ 2. $nums[i] - 1$ 3. $nums[j] = nums[i] - 1$ 4. $dp[i] = \max(dp[i], dp[j] + 1)$...

Good attempt. You got the state idea, but there's one important...

$nums[j] < nums[i]$, $dp[i] = \max(dp[i], dp[j] + 1)$

Exactly. ✓ For LIS: $dp[i] = \max(dp[i], dp[j] + 1)$ for every: $j < i$ and $nums[j] < nums[i]$...

donno about this one , point me in the right direction

No problem. Let's not jump to the recurrence yet. I'll guide you toward i...

$dp[i] = dp[i] \vee (dp[i - k] + 1)$ and $input_word[i - k : i] == word$ for...

You're very close. The core idea is right: choose a dictionary word as th...

next

Let's move to the next pattern from the material: Two-Sequence DP...

1. lcs of $text1[0...i - 1]$ and $text2[0...j - 1]$ 2. match or not 3. $dp[i - 1][j] - 1$...

Exactly right. Your LCS derivation is essentially correct. Just two small...
next

Thinking ● Located the next edit distance problem ✓ Worked for a coup...

1. min operations to convert word1[0..i-1] into word2[0...j-1] 2. no...

Yes - almost completely correct. There is just one important issue: you...
one thing i don't understand with respect to what you are classifying thi...

Yes - the confusing part is "insertion/deletion/replacement with respect ...
ok , next

Good. Next: Distinct Subsequences - this is the next Two-Sequence DP...
distinct subsequences of s[:i] matching t[:j]

Exactly. ✓ So: $dp[i][j]$ = number of distinct subsequences of s[:i] that...
1. option1 donno

Yes — option 1 is correct. For the matching case: $dp[i][j]=dp[i-1]...$
either use s[i - 1] to match the previous t[:j - 1] else ignore s[i - 1] so ,...

Yes — this is exactly the subtle point in Distinct Subsequences. Your...
ok , next

Thinking ● Identified the next dynamic programming topic ✓ Worked fo...
you forgot one thing , give the template codes for memo , table , for bo...
You're right - I should include the reusable code templates as we go. We...
ok , continue with the problems

Yes. We'll continue exactly from the material, and now move into Interv...
longest palindromic subsequence of s[i...j]

Exactly. ✓ So our state is: $dp[i][j]$ = length of the longest palindromic...
1. 1 2. 0

Exactly. ✓ For Longest Palindromic Subsequence (LPS): $i == j \rightarrow$ one...
leave it move to next question , also , dont give the answer

Got it — we'll skip solving that example, and I won't give answers to the...
first state the problem , i dont know matrix chain multiplication

Absolutely. Let's first understand the problem itself—no DP yet. Matrix...
D

Correct. ✓ D — Interval DP $\rightarrow$ Partition. Now let's derive the state. For...

donno

No problem. Let's make the state intuitive before asking you again. We...

minimum matrix multiplication cost for multiplything matrix i through j

Exactly. ✓ $dp[i][j]$ = minimum multiplication cost for multiplying matrix...

1 , 2 , 3 ?

Yes, exactly. ✓ For: $A_1 \times A_2 \times A_3 \times A_4$ the possible split points are: k ...

$dp[i][k]$, $dp[k + 1][j]$

Exactly. ✓ For a split at k: $A_i \dots A_k \mid A_{k+1} \dots A_j$ we solve: $dp[i][k] \rightarrow \dots$

$p * q * r$

Correct. ✓ If: $A = p \times q$ $B = q \times r$ then multiplying them costs: $p \times q \times \dots$

1. $dp[i][k] + dp[k + 1][j] + \text{cost } p \times q \times r$ explain why

Yes — $dp[i][k] + dp[k+1][j] + \text{cost}$ is correct. ✓ Now let's understand...

$10 * 20$, $20 * 30$, $30 * 40$

Correct. ✓ So: $A_1 = 10 \times 20$ $A_2 = 20 \times 30$ $A_3 = 30 \times 40$ The dimensio...

$10 * 30$

Exactly. ✓ For the split: $A_1 \times A_2 \mid A_3$ we have: $A_1 = 10 \times 20$ $A_2 = 20 \times 30$ So:...

$10 * 30 * 40$

Exactly. ✓ $10 \times 30 \times 40 = 12,000$ So for the split $k=2$: $10 \times 30 \mid A_2 \times A_3$ the...

why i - 1 ?

Good question. The i - 1 comes from how we store the matrix dimensio...

ok , next

Good. We've finished the Matrix Chain Multiplication example and the...

min sum along 0 , 0 to r , c

Yes. ✓ That's exactly the right state: $dp[r] \dots$

$\min(dp[r - 1][c] , dp[r][c - 1])$

Correct. ✓ Almost complete. You forgot to add the current cell's value....

$grid[0][0]$

Exactly. ✓ For the starting cell: $dp[0][0] = grid[0][0]$ Because there is no...

next

Good. Let's continue with the next Grid DP problem from the material:...

$1 + \min(dp[r - 1][c] , dp[r][c - 1])$

Almost! But this is the key difference between Minimum Path Sum and...

1

Correct. ✅ $dp[0][0]=1$ Because there is exactly one way to be at the...
next

Great. Let's continue with the next Grid DP problem from your material:...
largest square ending at r, c

Exactly. ✅ $dp[r][c]$ =largest square side length ending at (r,c) Now com...
donno

No problem. Let's derive it instead of memorizing it. Suppose: A B C X...
min

Exactly. Minimum is the key. 👍 Because the square can only be as larg...
why $1 +$, make sense intuitively

Yes — the $+1$ has a very intuitive meaning. Remember what: $dp[r][c]$...
next

Next, let's finish Maximal Square before moving to the next DP pattern....
this isn't needed

Yep. We don't need to spend time on that final-answer detail. Maximal...
ok , next

Thinking ● Found the next material after Grid Knapsack ✓ Worked for 7...
total should be even

Exactly. If the total is odd, equal partition is impossible. For: $[1, 5, 11, 5]$...
using the first i elements , can we sum to s

Exactly. ✅ So our state is: $dp[i]$...
take it or not

Exactly. 👍 For each element, there are two choices: Don't take it → can...
continue

Good. Let's continue with Partition Equal Subset Sum, following the...
next

Next: optimize the 2D Knapsack DP to 1D. Our 2D state was: $dp[i][s]$ Bu...
donno

No problem — this is an important detail. For 0/1 Knapsack, we go right...

wait how does processing right to left make sense , i mean , you would...

Ah — this is the subtle part. Your intuition is right: at the very beginning...

what happens when we go left to right ?

If we go left → right, the problem is that the dp array gets updated whil...

ok , where to use this and where not to

Use right → left specifically when you are doing 0/1 Knapsack — meanin...

ok , next

Next in the material is Tree DP. [dp-pattern-cheat-sheet.pdf](#) Tree DP The...

give an example , i have never learned this thing

Let's slow down and build Tree DP from zero, because it's different from...

Help me derive the two states

Absolutely. Let's derive them rather than memorize them. We have this...

code

Here is the standard Tree DP code for the two states we just derived:...

give the template code

For Tree DP, the reusable template is: `def dfs(node): if not node: return...`

next

Next: Bitmask DP This is the last major DP classification in your materia...

continue

Let's do the smallest possible Bitmask DP example first. Suppose we ha...

i dont understand , what is the problem exactly , what are we solving ?

You're right — I jumped into the implementation technique before...

continue

Now let's derive the TSP Bitmask DP state, exactly like we did for Tree D...

what are the two properties of dp

Thinking ● Searched source files for supporting material ✓ Worked for ...

what are the two properties of greedy

The two properties required for a greedy algorithm are: Greedy Choice...



dp-pattern-cheat-sheet.pdf

go through it completely , i want to learn , this , so , what is dp , how dp problems can be identified and according to this material , how dp problems can be classified , for now , explain these in detail , we'll dig deeper

12/09/2026, 16:19:49



Thinking

- **Identified DP definitions and classifications**

✓ Worked for 8s

Yes. I went through the DP material you uploaded, including the later sections where it refines the classification into **Prefix, Interval/Shrinking Interval, Partition, Two-Sequence, Grid, Knapsack, Tree, and Bitmask DP**. `dp-pattern-cheat-sheet.pdf`

For now, let's **not solve individual DP problems yet**. The goal is to build the mental model:

What is DP? → How do I recognize DP? → Once I know it's DP, how do I classify it?

That foundation will make the later patterns much easier.

1. What exactly is Dynamic Programming?

The simplest useful definition is:

Dynamic Programming is a technique for solving a problem by breaking it into smaller subproblems, solving each relevant subproblem once, and reusing those results.

There are three important ideas hidden inside that definition.

Example

Suppose I ask:

How many ways can I climb 5 stairs if I can take either 1 or 2 steps at a time?

To reach stair 5, I could come from:

- stair 4 → take 1 step
- stair 3 → take 2 steps

So:

$$\text{ways}(5) = \text{ways}(4) + \text{ways}(3)$$

But `ways(4)` itself needs `ways(3)` and `ways(2)`.

And `ways(3)` also needs `ways(2)`.

So we get:

```
ways(5)
├─ ways(4)
│   ├─ ways(3)
│   │   ├─ ways(2)
│   │   └─ ways(1)
│   └─ ways(2)
└─ ways(3)
    ├─ ways(2)
    └─ ways(1)
```

Notice:

```
ways(3) appears multiple times
ways(2) appears multiple times
ways(1) appears multiple times
```

These are **overlapping subproblems**.

Instead of recalculating them:

```
ways(3) → calculate  
ways(3) → calculate again  
ways(3) → calculate again
```

we calculate once:

```
ways(3) = 3
```

and remember it.

That's the core idea of DP.

2. Why is it called "Dynamic Programming"?

Don't get too attached to the name.

"Dynamic" doesn't mean the answer changes dynamically.

"Programming" doesn't originally mean writing code.

Think of it as:

Building a large answer from answers to smaller states.

The important concept is **state**.

3. The most important word in DP: STATE

A **state** represents one particular smaller version of the original problem.

For example:

```
dp[i]
```

might mean:

*"The answer for the first **i** elements."*

Or:

```
dp[i][j]
```

might mean:

"The answer for the first `i` elements of A and first `j` elements of B."

Or:

```
dp[l][r]
```

might mean:

"The answer for the interval from `l` to `r`."

The material repeatedly emphasizes this:

The shape of the state is what helps you classify the DP problem. `dp-pattern-cheat-sheet.pdf`

This is probably the **single most important idea from the entire material**.

4. DP is NOT simply "use an array called dp"

This is a common beginner misconception.

You don't look at a problem and say:

"I need an array, therefore DP."

Instead:

```
Problem
  ↓
Can I break it into smaller subproblems?
  ↓
Do smaller subproblems repeat?
  ↓
Can their answers be reused?
  ↓
DP may be appropriate
```

And then:

What makes one subproblem different from another?

↓

Define the STATE

↓

State shape gives PATTERN

The material explicitly recommends asking what makes one subproblem different: index, interval, two indices, grid cell, subset, tree node, etc. [dp-pattern-cheat-sheet.pdf](#)

5. How do I identify whether a problem is DP?

This is more important than memorizing patterns.

There isn't one magical sentence that says "this is DP."

Instead, look for a combination of clues.

Clue 1 - The problem asks for an optimum/count/possibility

Typical things:

Maximum

What is the maximum profit?

What is the longest subsequence?

What is the maximum score?

Minimum

What is the minimum number of coins?

What is the minimum cost?

What is the minimum number of operations?

Number of ways

How many ways can we reach the target?

How many ways can we decode this string?

Possibility

Is it possible to form this sum?

Can we reach the end?

These are **strong clues**, but they do NOT automatically mean DP.

For example:

Find the maximum element in an array.

That's optimization, but it's not DP.

So we need more.

6. Clue 2 - There are choices

DP problems frequently involve making decisions.

For example:

House Robber

At each house:

```
Rob it  
OR  
Don't rob it
```

Knapsack

For every item:

```
Take it  
OR
```

Don't take it

The material describes this as the hallmark of Knapsack DP. [dp-pattern-cheat-sheet.pdf](#)

Decode Ways

At each position:

Use 1 digit
OR
Use 2 digits

[dp-pattern-cheat-sheet.pdf](#)

Interval problems

You might ask:

What happens to the left end?
What happens to the right end?

or:

Where should I split?

So a very useful question is:

"What choices do I have?"

7. Clue 3 - Choices lead to smaller versions of the same problem

This is where DP becomes much more likely.

Suppose:

Original problem
↓
Choice

```
  ↙  ↘  
smaller smaller  
problem problem
```

For example, Climbing Stairs:

Reach stair 5

```
      /      \  
    from 4    from 3  
      ↓      ↓  
ways(4)     ways(3)
```

Both are the **same type of problem**, just smaller.

That's important.

8. Clue 4 - The smaller problems overlap

This is the big one.

Consider:

```
F(5)  
├─ F(4)  
│  ├─ F(3)  
│  └─ F(2)  
└─ F(3)  
    ├─ F(2)  
    └─ F(1)
```

F(3) and F(2) are repeated.

That's **overlapping subproblems**.

Without overlap, DP may not provide the same benefit.

9. Another important property: Optimal Substructure

You'll hear this phrase constantly in DP.

Optimal substructure means that an optimal solution to a problem can be constructed from optimal solutions to its smaller subproblems.

Example:

Suppose:

Best way to reach stair 5

comes from:

Best way to reach stair 4
+
one step

or:

Best way to reach stair 3
+
two steps

We don't need some weird non-optimal solution to stair 3 to produce the optimal solution to stair 5.

The smaller optimal answers are sufficient.

10. So the classic DP test

When you see a problem, mentally ask:

Question 1

Are there multiple choices / decisions?

Question 2

Do those choices create smaller versions of the same problem?

Question 3

Do those smaller problems repeat?

Question 4

Can I store their answers and reuse them?

If the answer is broadly **yes**, DP is a strong candidate.

The two theoretical properties to remember are:

```
Overlapping Subproblems
      +
Optimal Substructure
      ↓
DP candidate
```

11. But don't immediately write `dp[i]`

This is where many people go wrong.

Suppose the problem is:

Find the LCS of two strings.

You shouldn't immediately think:

```
dp[i]
```

because there are **two strings**.

You need information about both.

So:

```
dp[i][j]
```

might be appropriate.

The material's universal process is:

*First determine what makes one subproblem different from another. **That determines the state.*** `dp-pattern-cheat-sheet.pdf`

This leads us to classification.

12. How DP problems are classified

The material's most useful idea is:

Don't classify primarily by problem name. Classify by the shape of the subproblem/state.

For example:

```
dp[i]
```

suggests one-dimensional/prefix-style DP.

```
dp[i][j]
```

could mean several completely different things.

It might be:

```
two strings
  ↓
Two-Sequence DP
```

or:

```
one array, interval [i,j]
  ↓
Interval DP
```

or:

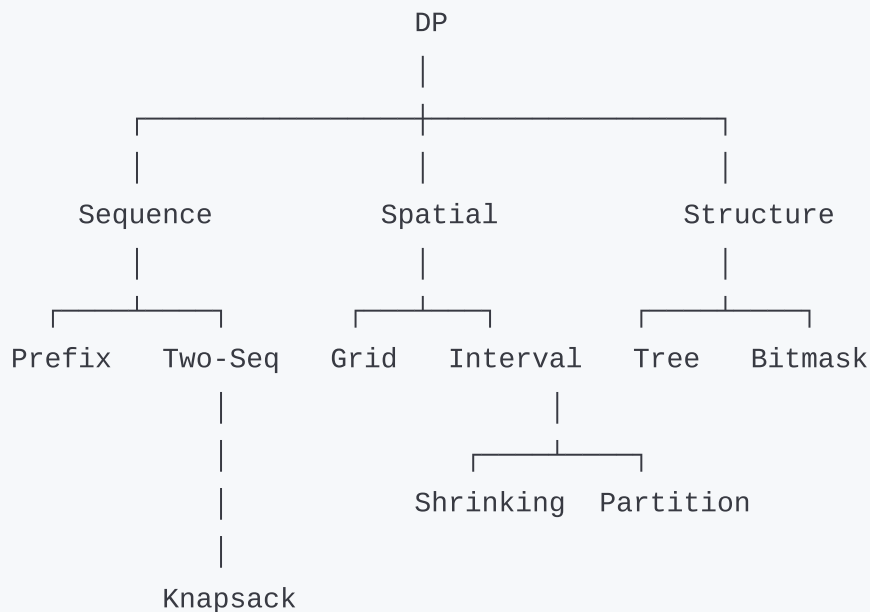
```
grid[r][c]
  ↓
Grid DP
```

So `dp[i][j]` **alone isn't enough**.

You need to understand what `i` and `j` mean.

13. The classification hierarchy

According to the material, the main patterns are:



I'd actually think of it slightly differently:

Pattern	Core state idea	Main question
Prefix DP	<code>dp[i]</code>	What is the answer up to <code>i</code> ?
Sorted Prefix DP	<code>dp[i]</code>	What is the best answer ending at <code>i</code> after ordering?
Two-Sequence DP	<code>dp[i][j]</code>	What happens to the two prefixes?
Interval DP	<code>dp[l][r]</code>	What happens inside this range?
Partition DP	<code>dp[l][r] + split k</code>	Where should I split?
Grid DP	<code>dp[r][c]</code>	Where could I have come from?
Knapsack DP	<code>dp[i][target]</code>	Take or don't take?
Tree DP	<code>dp[node]</code>	What information comes from children?

Pattern	Core state idea	Main question
Bitmask DP	<code>dp[mask]</code>	Which elements have already been used?

This classification is directly reflected in the material. `dp-pattern-cheat-sheet.pdf`

14. Pattern 1 - Prefix DP

Imagine an array:

```

index:  0   1   2   3   4   5
        ───────────
array:  A   B   C   D   E   F

```

Prefix DP asks:

What is the answer using the first `i` elements?

So:

```
dp[i]
```

might mean:

```
answer for A[0 ... i]
```

The material's examples include:

- Climbing Stairs
- House Robber
- Decode Ways
- Coin Change
- Word Break
- LIS variants

Recognition

Think:

"I'm processing a sequence from left to right."

And ask:

How could I have arrived at this position?

For example:

```
i
↑
could come from i-1
could come from i-2
could come from some earlier j
```

That's why the material gives the checklist:

```
i-1?
i-2?
any j < i?
```

[dp-pattern-cheat-sheet.pdf](#)

15. Pattern 2 - Sorted Prefix DP

This is a variation of Prefix DP.

You first create a useful ordering:

```
sort
↓
process left → right
↓
dp[i]
```

The state is often:

best answer ending at element `i`.

Example: LIS-style problems.

Conceptually:

```
           previous valid element
                ↓
1 → 3 → 5 → 8
    ↓
    dp[i]
```

You ask:

Which previous element can connect to the current element?

The material gives examples such as:

- LIS variants
- Box Stacking
- Job Scheduling
- Longest Chain of Pairs
- Russian Doll Envelopes

16. Pattern 3 - Two-Sequence DP

Now suppose you have:

```
A = ABCDE
B = ACDF
```

You're comparing **two sequences**.

The natural state becomes:

```
dp[i][j]
```

meaning:

Answer using the first `i` elements of A and first `j` elements of B.

This is the classic pattern for:

- LCS
- Edit Distance
- Distinct Subsequences

`dp-pattern-cheat-sheet.pdf`

The key question becomes:

What happens to the last elements/characters of the two sequences?

For LCS:

```
A ... X
B ... X
```

If they match:

```
take both
  ↓
dp[i-1][j-1] + 1
```

If they don't:

```
discard A's character
OR
discard B's character
```

Hence:

```
max(
    dp[i-1][j],
    dp[i][j-1]
)
```

`dp-pattern-cheat-sheet.pdf`

So the mental trigger is:

Two sequences + comparison/alignment/matching/transformation → Two-Sequence DP.

17. Pattern 4 - Interval DP

Now imagine:

```
A B C D E F
↑       ↑
l       r
```

Instead of asking about a prefix, you're solving a **range**.

```
dp[l][r]
```

means:

Answer for the interval `[l, r]`.

The material gives examples:

- Palindrome problems
- Burst Balloons
- Matrix Chain Multiplication
- Palindrome Partitioning

`dp-pattern-cheat-sheet.pdf`

Recognition clues:

```
substring
subarray
range
interval
[i, j]
```

The important question is:

What happens to the ends?

For example, Longest Palindromic Subsequence:

```
a ..... a
↑           ↑
l           r
```

If:

```
s[l] == s[r]
```

then:

```
2 + dp[l+1][r-1]
```

Otherwise:

```
discard left
OR
discard right
```

This is **Shrinking Interval DP**. [dp-pattern-cheat-sheet.pdf](#)

18. Partition DP - the important distinction

This is the one you specifically asked about earlier.

Partition DP is absolutely part of the material.

It is often treated as a **subtype of Interval DP**, but it's useful enough to learn separately because its transition has a distinctive structure. [dp-pattern-cheat-sheet.pdf](#)

Suppose:

```
A B C D E
```

and you're solving:

```
dp[l][r]
```

Instead of asking:

What happens to the ends?

you ask:

Where should I split this interval?

For example:

```
A B | C D E
    ↑
    k
```

or:

```
A B C | D E
      ↑
      k
```

or:

```
A B C D | E
        ↑
        k
```

You try every possible `k`.

So the recurrence looks like:

$$dp[l][r] = \text{best}_k (dp[l][k] + dp[k+1][r] + \text{cost})$$

`dp-pattern-cheat-sheet.pdf`

The recognition question is extremely powerful:

"Where should I split?"

or

"Where should I cut?"

or

"Which partition gives the best answer?"

→ Think **Partition DP**.

The material specifically identifies:

- Matrix Chain Multiplication
- Burst Balloons
- Minimum Cost to Cut a Stick

as classic examples. [dp-pattern-cheat-sheet.pdf](#)

19. Shrinking Interval vs Partition DP

This distinction is worth remembering.

Shrinking Interval

[1 r]

↓ ↓

[1+1 .. r-1]

Question:

What happens to the two ends?

Example:

Longest Palindromic Subsequence

Partition

[1 r]

↓
k

[1 k] [k+1 r]

Question:

Where should I split?

Example:

Matrix Chain Multiplication

The material explicitly makes this distinction. [dp-pattern-cheat-sheet.pdf](#)

And importantly:

A problem can belong to both.

Burst Balloons is a good example:

Interval DP
because state = dp[l][r]

Partition DP
because we try every split/last choice k

[dp-pattern-cheat-sheet.pdf](#)

So these classifications aren't always mutually exclusive.

20. Pattern 5 - Grid DP

Now imagine:

```
    c
  0 1 2 3
0 . . . .
```

```
r 1 . . . .  
  2 . . . .  
  3 . . . .
```

State:

```
dp[r][c]
```

means something about cell (r, c) .

The key question:

Where could I have come from?

For example, if movement is only:

```
→  
↓
```

then to reach:

```
(r, c)
```

you could have come from:

```
(r-1, c)    ↑  
(r, c-1)    ←
```

Therefore:

$$dp[r][c] = dp[r-1][c] + dp[r][c-1]$$

for Unique Paths. [dp-pattern-cheat-sheet.pdf](#)

Typical examples:

- Unique Paths
- Minimum Path Sum
- Maximal Square
- Dungeon Game

dp-pattern-cheat-sheet.pdf

Recognition:

```
matrix
grid
board
cells
paths
movement
up/down/left/right/diagonal
```

→ Think **Grid DP**.

21. Pattern 6 - Knapsack DP

This one has a very recognizable decision:

Take or don't take?

Suppose:

```
items = [2, 3, 7, 8]
target = 10
```

For every item:

```
      item
     /   \
    TAKE  DON'T TAKE
```

The state might be:

```
dp[i][sum]
```

meaning:

*Using the first **i** items, can I make **sum** ?*

The material describes exactly this interpretation. dp-pattern-cheat-sheet.pdf

Typical problems:

- 0/1 Knapsack
- Subset Sum
- Partition Equal Subset Sum
- Target Sum
- Coin Change

dp-pattern-cheat-sheet.pdf

Recognition:

```
subset
sum
target
capacity
weight/value
choose items
partition
```

→ Think **Knapsack DP**.

22. Pattern 7 - Tree DP

Now the problem isn't a sequence.

It's a tree:



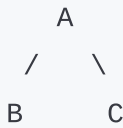
State:

```
dp[node]
```

The key question becomes:

What information do I need from my children?

For example, House Robber III:



If I rob A:

I cannot rob B or C

If I don't rob A:

I can choose whether to rob B/C

So we may need two states:

```
dp[node][rob]
dp[node][not_rob]
```

The material describes this exact idea. [dp-pattern-cheat-sheet.pdf](#)

Recognition:

```
tree
node
subtree
children
parent
```

→ Think **Tree DP**.

Usually computation happens bottom-up:

```
children
  ↓
parent
  ↓
root
```

which is why the material recommends **postorder DFS**. [dp-pattern-cheat-sheet.pdf](#)

23. Pattern 8 - Bitmask DP

This is usually for relatively small `n`, where we care about **which elements have already been used**.

Suppose:

A B C D

A bitmask can represent:

```
0000 → nothing used
0001 → A used
0011 → A, B used
1011 → A, B, D used
1111 → all used
```

So:

`dp[mask]`

represents some answer for the subset represented by `mask`.

The key question:

Which elements have already been used?

Then:

Which unused element should I choose next?

The material gives examples:

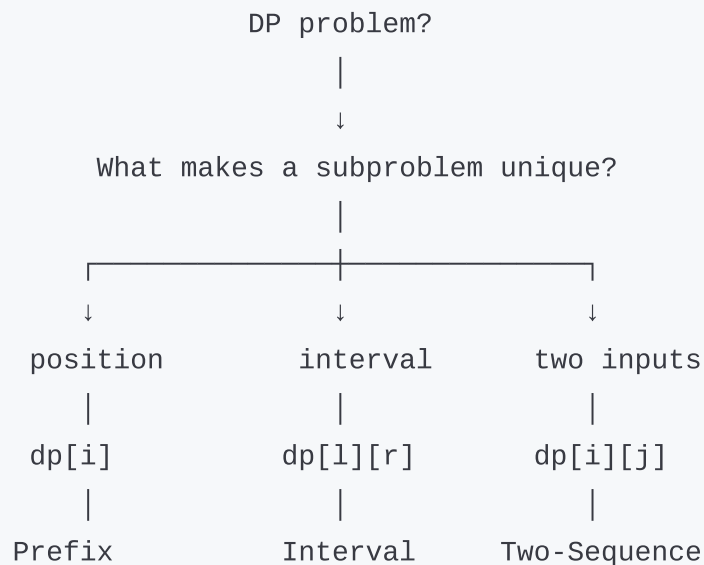
- Travelling Salesman
- Assignment Problem
- Shortest Hamiltonian Path

[dp-pattern-cheat-sheet.pdf](#)

24. The classification process you should actually use

Don't memorize the table blindly.

When you get a new problem, do this:



Then:

Is it a grid?



Grid DP

Are we selecting items toward a target?



Knapsack DP

Is it a tree?



Tree DP

Is the state about used subsets?



Bitmask DP

And for intervals:

```

dp[l][r]
  |
  |— What happens to ends?
  |   ↓
  |   Shrinking Interval
  |
  |— Where should I split?
      ↓
      Partition DP

```

This is essentially the "real secret" emphasized by the material:

Don't memorize problem names. Memorize the shape of the subproblem.

[dp-pattern-cheat-sheet.pdf](#)

25. One extremely important correction

Don't think:

`dp[i]` = Prefix DP

`dp[i][j]` = Two-Sequence DP

That is **too simplistic**.

Instead think:

`dp[i]`

Ask:

What does `i` represent?

Could be:

```

prefix position
ending position
amount
time
etc.

```

`dp[i][j]`

Ask:

What do `i` and `j` represent?

They could represent:

two sequences → Two-Sequence DP

left/right interval → Interval DP

row/column → Grid DP

item/target → Knapsack DP

The meaning of the indices determines the pattern, not the number of indices.

That's a very important distinction.

26. The ultimate DP mental model

I want you to eventually look at a problem like this:

"Given an array, choose elements to maximize the sum, but you cannot choose adjacent elements."

and your brain should NOT immediately say:

"Oh, House Robber. I memorized this."

Instead:

What makes a subproblem different?

↓

Position `i`

↓

`dp[i]`

↓

What is the last decision?

↓

Take `nums[i]`

OR

Don't take `nums[i]`

↓

If take → previous cannot be `i-1`

If don't → previous can be `i-1`

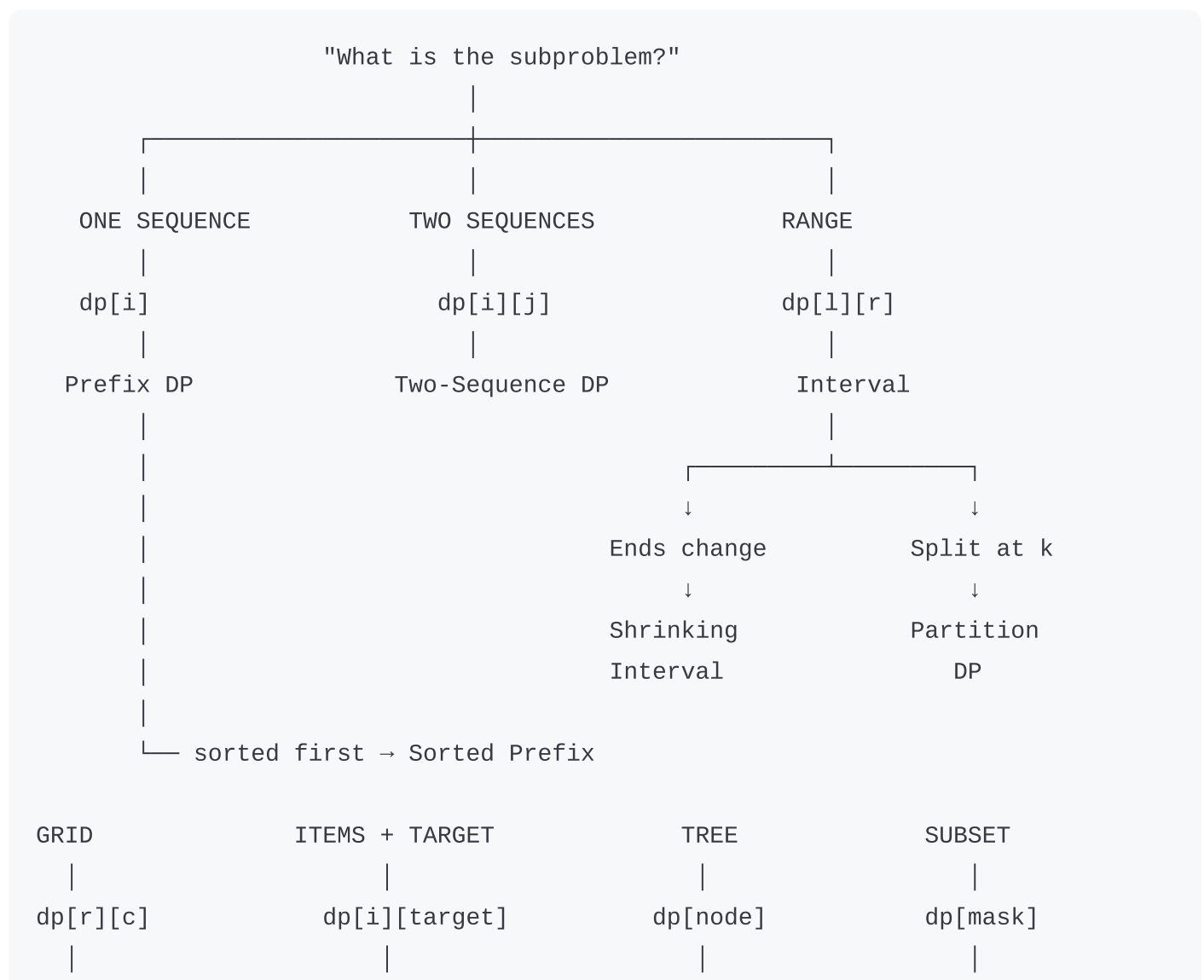
↓

derive recurrence

That's exactly the skill the material wants you to develop: **state first, last decision second, recurrence third.** [dp-pattern-cheat-sheet.pdf](#)

27. Your DP recognition cheat sheet

Keep this in your head:



Grid DP

|

"Where did I
come from?"

Knapsack DP

|

"Take or don't
take?"

Tree DP

|

"What do I need
from children?"

Bitmask DP

|

"What is
already used?"

This captures the classification used throughout your material. [dp-pattern-cheat-sheet.pdf](#)

28. One final distinction: DP ≠ memoization

You will encounter two ways to implement DP:

Top-down

```
recursive thinking
+
memoization
```

You start with the final problem and recursively ask for smaller states.

Bottom-up

```
start with small states
↓
build larger states
```

using a table.

Both are **DP**.

The material focuses heavily on deriving the **state and recurrence first**, rather than jumping into code. [dp-pattern-cheat-sheet.pdf](#)

So our learning order should be:

1. What is DP? ← DONE
2. How to recognize DP? ← DONE
3. How to classify DP? ← DONE
4. What exactly is a STATE?
5. How to identify the LAST DECISION?
6. How to derive a recurrence?

7. Base cases
8. Top-down vs bottom-up
9. Solve each pattern systematically
10. Mixed/harder DP problems

The next thing I'd recommend digging into is `STATE` + `LAST DECISION`, **because those two ideas are the bridge between "I know the DP pattern" and "I can actually solve a new DP problem."** The material itself calls the last decision the most important question in the derivation process. `dp-pattern-cheat-sheet.pdf`

12/09/2026, 16:19:53



we'll go pattern by pattern starting with prefix , explain it , give a template code for it in python for both memo and table version

12/09/2026, 16:34:35



Yes. We'll go **pattern by pattern**, and for each one I'll build the pattern from the ground up rather than just giving you code.

According to your material, **Prefix DP** is the first major pattern. The key question is:

"Can I define the answer using only the first i elements?"

Then the state is usually `dp[i]`. [dp-pattern-cheat-sheet.pdf](#)

1. Prefix DP

The basic idea

Imagine an array:

```
index:   0   1   2   3   4   5
         ↓   ↓   ↓   ↓   ↓   ↓
nums = [A,  B,  C,  D,  E,  F]
```

Prefix DP processes this **from left to right**.

Instead of solving the entire problem at once, we ask:

What is the answer for the prefix ending at i ?

For example:

```
i = 0 → [A]
i = 1 → [A, B]
i = 2 → [A, B, C]
```

```
i = 3 → [A, B, C, D]
...
```

So we create:

```
dp[i]
```

where `dp[i]` represents some answer associated with the first `i` elements or with position `i`, depending on the problem. Your material gives both forms:

```
dp[i] = answer using the first i elements
```

or

```
dp[i] = best answer ending at position i
```

`dp-pattern-cheat-sheet.pdf`

The **exact meaning** of `dp[i]` is something we must define ourselves for each problem.

2. Why is this DP?

Take the classic **Climbing Stairs** problem:

You are at stair 0. You can climb either 1 or 2 stairs. How many ways can you reach stair `n`?

Suppose:

```
n = 5
```

To reach stair 5, your last move must have been:

```
from 4 → 5
```

or

```
from 3 → 5
```

Therefore:

```
ways(5)
  /   \
ways(4) ways(3)
```

And both of those themselves break down:

```
      ways(5)
      /     \
    ways(4)   ways(3)
    /  \     /  \
ways(3) ways(2) ways(2) ways(1)
  ↑           ↑
  └── repeated ┘
```

`ways(3)` and `ways(2)` are calculated multiple times.

DP says:

Calculate each smaller problem once and remember its answer.

3. The most important Prefix DP thought process

Your material gives a very useful procedure. `dp-pattern-cheat-sheet.pdf`

For every Prefix DP problem, ask:

Step 1 - Define the state

What does `dp[i]` mean?

Step 2 - Find the last action

What was the last thing I did before reaching `i`?

Step 3 - Find previous states

From which previous states could I have reached `i`?

Step 4 - Combine them

Depending on the problem:

```
maximum → max(...)
minimum → min(...)
count    → sum(...)
boolean  → OR / AND
```

Step 5 - Base cases

What happens at the smallest possible prefixes?

4. Example: House Robber

This is probably the best example for understanding the pattern.

Problem:

```
nums = [2, 7, 9, 3, 1]
```

You cannot rob two adjacent houses.

We want the maximum money.

Step 1: Define `dp[i]`

Ask:

What does `dp[i]` represent?

We'll define:

```
dp[i] = maximum money we can rob from houses 0...i
```

So:

```
dp[0] = answer for [2]
dp[1] = answer for [2,7]
dp[2] = answer for [2,7,9]
...
```

This is exactly the Prefix DP shape described in the material. [dp-pattern-cheat-sheet.pdf](#)

Step 2: What was the last decision?

We're standing at house `i`.

There are only two possibilities:

Option A - Don't rob house `i`

Then the answer is simply:

```
dp[i-1]
```

because we've already solved houses `0...i-1`.

Option B - Rob house `i`

If we rob house `i`, we **cannot rob** `i-1`.

So the previous usable state is:

```
dp[i-2]
```

Then add the money from house `i`:

```
dp[i-2] + nums[i]
```

Therefore:

```
dp[i] = max(  
    dp[i-1],  
    dp[i-2] + nums[i]  
)
```

This is the exact kind of recurrence your material highlights. [dp-pattern-cheat-sheet.pdf](#)

5. Prefix DP recurrence shape

Notice something important.

The recurrence isn't always literally:

```
dp[i] = ...
```

The **structure** is:

```
dp[i]
  ↓
look at possible previous states
  ↓
combine them
```

For example:

Climbing Stairs

```
dp[i] = dp[i-1] + dp[i-2]
```

Because we're **counting** ways. [dp-pattern-cheat-sheet.pdf](#)

House Robber

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i])
```

Because we're maximizing.

Coin Change

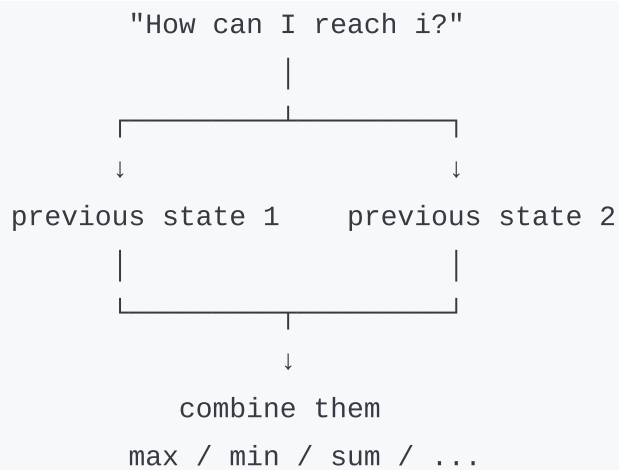
```
dp[i] = min(dp[i-c] + 1 for c in coins)
```

Because we're minimizing. [dp-pattern-cheat-sheet.pdf](#)

So a useful mental model is:

PREFIX DP

```
dp[i]
|
```



6. Memoization vs Tabulation

There are two ways to implement the same DP.

Your material eventually uses the generic bottom-up form:

```
dp = [0] * n

# initialize base cases

for i in range(start, n):
    dp[i] = recurrence

return dp[n-1]
```

[dp-pattern-cheat-sheet.pdf](#)

But let's understand both properly.

7. Template 1 - Memoization / Top-Down

Memoization means:

Start with the original problem and recursively break it down, while caching answers you've already calculated.

Generic Prefix DP template:

```
def solve(nums):
    n = len(nums)
```

```

memo = {}

def dp(i):
    # Base case
    if i == 0:
        return ...

    if i in memo:
        return memo[i]

    # Recurrence
    ans = ...

    memo[i] = ans
    return ans

return dp(n - 1)

```

The important structure is:

```

def dp(i):

    if base_case:
        return base_value

    if i in memo:
        return memo[i]

    # calculate from smaller states
    ans = ...

    memo[i] = ans
    return ans

```

Think of it as:

```

solve(n)
  ↓
solve(n-1)
  ↓

```

```
solve(n-2)
  ↓
...
```

but whenever we encounter something we've already solved:

```
"Already calculated → return cached answer"
```

8. Template 2 - Tabulation / Bottom-Up

Tabulation does the opposite.

Instead of starting at `n` and going backward, we start from the smallest state and build upward.

```
def solve(nums):
    n = len(nums)

    dp = [0] * n

    # Base cases
    dp[0] = ...

    if n > 1:
        dp[1] = ...

    # Build from smaller → larger
    for i in range(2, n):
        dp[i] = ...

    return dp[n - 1]
```

Conceptually:

```
dp[0]
  ↓
dp[1]
  ↓
dp[2]
  ↓
```

```
dp[3]
  ↓
dp[4]
  ↓
...
dp[n-1]
```

9. Same problem in both forms

Let's use **House Robber** because it clearly shows the difference.

Memoization

```
def rob(nums):
    n = len(nums)
    memo = {}

    def dp(i):
        if i < 0:
            return 0

        if i in memo:
            return memo[i]

        skip = dp(i - 1)
        take = dp(i - 2) + nums[i]

        memo[i] = max(skip, take)

        return memo[i]

    return dp(n - 1)
```

Notice the thought process:

```
dp(i)
|
├─ don't take i → dp(i-1)
```

```
|  
└─ take i      → dp(i-2) + nums[i]
```

Tabulation

```
def rob(nums):  
    n = len(nums)  
  
    if n == 0:  
        return 0  
  
    if n == 1:  
        return nums[0]  
  
    dp = [0] * n  
  
    dp[0] = nums[0]  
    dp[1] = max(nums[0], nums[1])  
  
    for i in range(2, n):  
        dp[i] = max(  
            dp[i - 1],  
            dp[i - 2] + nums[i]  
        )  
  
    return dp[n - 1]
```

Same recurrence.

The only real difference is **how we evaluate the states**.

10. The universal templates you should remember

Don't memorize the House Robber code.

Memorize these **structures**.

Memoization

```
def solve(input):  
    memo = {}
```

```
def dp(i):
    if base_case:
        return base_value

    if i in memo:
        return memo[i]

    ans = recurrence_using_smaller_states

    memo[i] = ans
    return ans

return dp(start_state)
```

Tabulation

```
def solve(input):
    n = ...

    dp = [initial_value] * n

    # Base cases
    dp[0] = ...
    dp[1] = ...

    # Build smaller → larger
    for i in range(..., n):
        dp[i] = recurrence_using_previous_states

    return dp[...]
```

11. One important variation: previous *any* index

Prefix DP doesn't necessarily mean only:

```
i-1
i-2
```

Sometimes `i` can depend on **any previous** `j < i`.

Your material explicitly points this out. `dp-pattern-cheat-sheet.pdf`

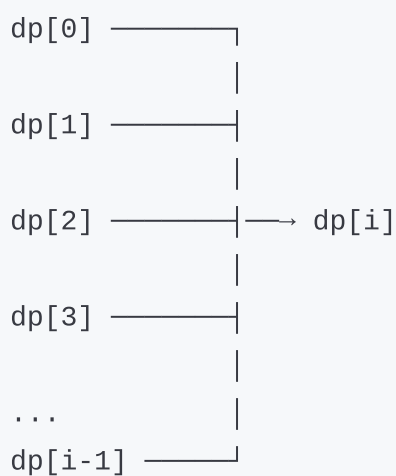
For example, LIS:

```
dp[i] = 1 + max(dp[j])
```

for all valid:

```
j < i
```

So the general picture becomes:



This is still **Prefix DP** because the state is still based on earlier positions:

```
dp[i]
```

and we're processing the sequence left → right.

12. How to recognize Prefix DP in an interview

When you see an array/string problem, ask:

Question 1

Can I describe the problem using the first `i` elements?

If yes, consider:

```
dp[i]
```

Question 2

Can I process the input from left → right?

Strong clue for Prefix DP. `dp-pattern-cheat-sheet.pdf`

Question 3

What was the last action before reaching i ?

This is often the key to the recurrence.

Question 4

Where could I have come from?

Possibilities might be:

```
i-1
i-2
...
any j < i
```

Question 5

Am I trying to maximize, minimize, count, or determine possibility?

That tells you how to combine the possibilities:

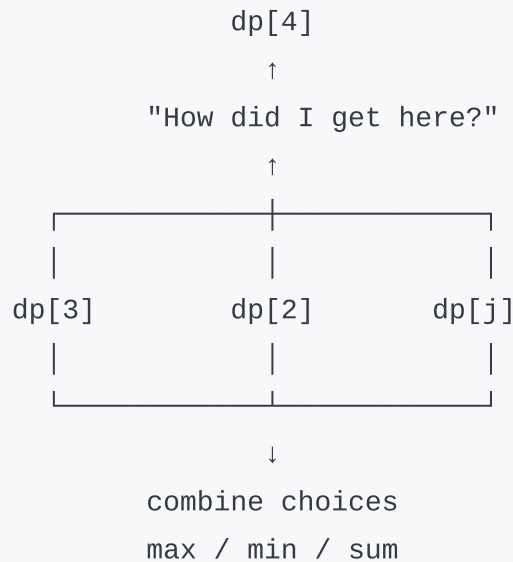
```
maximize → max()
minimize → min()
count    → sum()
possible → OR / AND
```

13. Prefix DP in one mental picture

If you remember only one diagram, remember this:

PREFIX DP

Input: A B C D E
 ↑ ↑ ↑ ↑ ↑
 0 1 2 3 4



The **core idea is not the array** `dp`.

It is:

Define the answer for a prefix/state `i`, identify the possible previous states, and reuse their answers.

That's why the material says the easiest way to classify a DP problem is to ask **“What does my state represent?”** `dp-pattern-cheat-sheet.pdf`

Prefix DP cheat sheet

Component	Typical form
State	<code>dp[i]</code>
Meaning	Answer for prefix / ending at <code>i</code>
Direction	Left → right
Main question	“How can I reach <code>i</code> ?”

Component	Typical form
Previous states	<code>i-1</code> , <code>i-2</code> , or any <code>j < i</code>
Combination	<code>max</code> , <code>min</code> , <code>sum</code> , boolean
Top-down	Recursion + <code>memo</code>
Bottom-up	Array + <code>for</code> loop
Examples	Climbing Stairs, House Robber, Decode Ways, Coin Change, LIS

`dp-pattern-cheat-sheet.pdf`

Next, the important thing is not another template. We should take 2-3 Prefix DP problems and practice **deriving** `dp[i]` **ourselves**-especially learning how to identify the **last decision**, because that is what actually lets you solve unfamiliar DP problems rather than memorize them.

12/09/2026, 16:34:37



let's solve some problem on this , just state the problem , i'll try to find the recurrence , dont say the answer make me derive it

12/09/2026, 16:39:56



Problem 1 — Climbing Stairs

You are climbing a staircase with n steps.

You can climb either **1 step or 2 steps** at a time.

Given n , return the **number of distinct ways** you can reach the top.

Example:

Input: $n = 5$

Output: 8

Don't write code yet.

Derive:

1. What does $dp[i]$ represent?
2. What is the **last action** before reaching stair i ?
3. From which previous states can you reach i ?
4. Therefore, what is the recurrence?
5. What are the base cases?

Give me your reasoning + recurrence. Don't worry about implementation yet.

12/09/2026, 16:39:57



1. ways to climb upto the stair i 2. jump 3. $i - 1$, $i - 2$
4. $dp[i] = dp[i - 1] + dp[i - 2]$ 5. $dp[0] = 1$, $dp[1] = 1$
, we'll not implement code , move with the next problem

12/09/2026, 16:42:07



Exactly right. One small wording improvement:

- **State:** $dp[i]$ = number of ways to reach stair i
- **Last action:** the final jump is either 1 step or 2 steps
- **Previous states:** $i - 1$, $i - 2$
- **Recurrence:** $dp[i] = dp[i - 1] + dp[i - 2]$
- **Base cases:** $dp[0] = 1$, $dp[1] = 1$

Good. Let's move on.

Problem 2 — House Robber

You are given an array `nums` , where `nums[i]` is the amount of money in the i -th house.

You want to rob the **maximum amount of money**, but you **cannot rob two adjacent houses**.

Example

```
nums = [2, 7, 9, 3, 1]
```

```
Output = 12
```

Because you can rob:

```
2 + 9 + 1 = 12
```

Your task is **only to derive the DP**, not code.

Answer these:

1. What does `dp[i]` represent?
2. What are the possible choices when considering house `i`?
3. If you **don't rob** house `i`, what previous state do you get?
4. If you **do rob** house `i`, what previous state can you use?
5. What is the recurrence?
6. What are the base cases?

Don't look for the answer mechanically—think about the last decision made at house `i`.

12/09/2026, 16:42:08



1. maximum amount of money stolen considering upto house i 2. rob it or not 3. $dp[i - 1]$ 4. $dp[i - 2] + \text{nums}[i]$ 5. $dp[i] = \max(dp[i - 2] + \text{nums}[i], dp[i - 1])$ 6. $dp[0] = \text{nums}[0]$

12/09/2026, 16:45:23



Perfect. Your recurrence is correct.

Only one detail to add: for the recurrence to work from $i = 2$, you also need:

- $dp[0] = \text{nums}[0]$
- $dp[1] = \max(\text{nums}[0], \text{nums}[1])$

Your reasoning for the transition is exactly what I wanted: **at house i , the last decision is either rob or don't rob.**

Let's make it slightly harder.

Problem 3 — Decode Ways

You are given a string containing digits. Each digit/number can represent a letter:

1 → A
2 → B
...
26 → Z

Given a string s , return the **number of ways to decode it.**

Example

$s = "226"$

Possible decodings:

2 2 6 → B B F

22 6 → V F

```
2 26 → B Z
```

```
Output = 3
```

Another example:

```
s = "06"
```

```
Output = 0
```

Because `0` cannot be decoded by itself.

Your job

Don't code. Derive the DP:

1. What does `dp[i]` represent?
2. What is the **last action** when decoding the first `i` characters?
3. What previous states can each action come from?
4. Under what conditions is each action valid?
5. What is the recurrence?
6. What are the base cases?

This one is important because it teaches you how DP transitions can depend on **validity conditions**, not just fixed indices.

12/09/2026, 16:45:24



1. ways to decode string $s[0...i]$
2. splitting the string
3. $dp[i - 1]$, $dp[i - 2]$
4. $dp[i - 1]$ is always valid , $dp[i - 2]$ is valid when $10 \leq s[i - 1:i] \leq 26$
5. $dp[i] = dp[i - 1]$; $dp[i] += dp[i - 2]$ if $10 \leq s[i - 1:i] \leq 26$
6. $dp[0] = 1$ $dp[1] = dp[0]$

12/09/2026, 16:55:09



Yes — **your recurrence idea is correct**, and you're starting to see the important DP pattern.

There are just **two important corrections**:

1. $dp[i-1]$ is not always valid

The one-character decoding is valid only if:

```
s[i-1] != '0'
```

For example:

```
"06"
```

The `0` cannot be decoded by itself, so we **cannot** automatically add $dp[i-1]$.

2. Two-character condition

For the last two characters, you want:

```
10 <= int(s[i-2:i]) <= 26
```

Notice the indices: if $dp[i]$ represents the first i characters, the last two characters are $s[i-2:i]$.

So conceptually:

```
decode last 1 digit  
/
```

```
dp[i] = dp[i-1] +  
        \  
        decode last 2 digits
```

with each contribution included **only when that decoding is valid**.

Your `dp[0] = 1` is also correct: there is **one way to decode an empty string**.

Your `dp[1]` needs to depend on whether the first character is `0`.

You got the main idea. 👍

Problem 4 — Coin Change

Given an array of coin denominations and an amount, return the **minimum number of coins** needed to make that amount.

You can use each coin **unlimited times**.

Example:

```
coins = [1, 2, 5]  
amount = 11  
  
Output = 3
```

Because:

```
5 + 5 + 1 = 11
```

Derive it:

1. What does `dp[i]` represent?
2. What is the **last action** when making amount `i`?
3. If the last coin chosen is `c`, what previous state do you reach?
4. What choices do you need to consider?
5. What is the recurrence?
6. What should the base case `dp[0]` be?

Again, **no code and no answer from me yet**.

12/09/2026, 16:55:11



1. min coins to amount to amount i 2. adding a coin to the previous amount 3. dp[i - c] 4. dp[i - c] for c in coins 5. min(dp[i - c]) for c in coins 6. dp[0] = 0 , for each coin c in coins dp[c] = 1 then give the correct complete recurrence for decode ways then

12/09/2026, 16:59:38



Your Coin Change derivation is **almost correct**, but there is one missing `+1`.

If the last coin chosen is `c`, then:

- `dp[i-c]` = minimum coins to make the previous amount
- `+1` = the coin `c` that we just added

So the recurrence is:

$$dp[i] = \min_{c \in \text{coins}, c \leq i} (dp[i - c] + 1)$$

And `dp[0] = 0`.

You don't actually need to explicitly initialize `dp[c] = 1`; that falls out of the recurrence when `i = c`.

Complete Decode Ways recurrence

Your idea was right. Let's write it precisely using the material's `dp[i] = number of ways to decode the first i characters` definition.

For each `i`:

Case 1 — Decode the last character alone

The last character is:

```
s[i-1]
```

It is valid if:

```
s[i-1] != '0'
```

If valid, it contributes:

$$dp[i - 1]$$

because we decode the first `i-1` characters and then decode this one character.

Case 2 — Decode the last two characters together

The two-character number is:

```
s[i-2:i]
```

It is valid if:

$$10 \leq \text{int}(s[i - 2 : i]) \leq 26$$

If valid, it contributes:

$$dp[i - 2]$$

Complete recurrence

Therefore:

$$dp[i] = \begin{cases} dp[i - 1], & \text{if } s[i - 1] \neq '0' \\ 0, & \text{otherwise} \end{cases} + \begin{cases} dp[i - 2], & \text{if } 10 \leq \text{int}(s[i - 2 : i]) \leq 26 \\ 0, & \text{otherwise} \end{cases}$$

with:

$$dp[0] = 1$$

and for a one-character string:

$$dp[1] = \begin{cases} 1 & s[0] \neq '0' \\ 0 & s[0] = '0' \end{cases}$$

The important DP insight

This problem follows exactly the "**last action**" method you used:

```

        last 1 digit
        /   → dp[i-1]
dp[i] ← decoding
        \
        last 2 digits
        → dp[i-2]
```

The only extra difficulty is that **each transition has a validity condition**.

Your Coin Change problem is now the next useful step because it introduces a different kind of Prefix DP: instead of only looking at `i-1 / i-2`, you look at **any previous amount** `i-c`.

12/09/2026, 16:59:40

[next](#)

12/09/2026, 17:01:06



Problem 5 — Longest Increasing Subsequence (LIS)

Given an array `nums`, find the length of the **longest strictly increasing subsequence**.

A subsequence does **not** need to be contiguous, but the relative order must be preserved.

Example:

```
nums = [10, 9, 2, 5, 3, 7, 101, 18]
```

```
Output = 4
```

One valid LIS is:

```
2 → 3 → 7 → 101
```

Derive the DP

1. What does `dp[i]` represent?
2. For an increasing subsequence ending at `nums[i]`, which previous elements can you extend?
3. What condition must `nums[j]` satisfy?
4. What previous states do you consider?
5. What is the recurrence?
6. What is the base case?

Important: this is the material's Prefix DP case where the transition can come from **any** `j < i`, not just `i-1` or `i-2`.

No code, and don't worry about the optimized $O(n \log n)$ LIS algorithm yet.

12/09/2026, 17:01:08



1. LIS ending at `nums[0...i]`
2. `nums[i] - 1`
3. `nums[j]`
`= nums[i] - 1`
4. `dp[j]` where `nums[j] == nums[i] - 1`
5. for `j` in `range(0 ... i)` where `nums[j] == nums[i] - 1`
`dp[i] = dp[j] + 1` if no such `j`, `dp[i] = 1`

12/09/2026, 17:05:33



Good attempt. You got the **state idea**, but there's one important conceptual mistake in the transition.

Your state

`dp[i]` = LIS ending at `nums[i]`

✓ Correct.

The mistake: previous element doesn't have to be `nums[i] - 1`

For a **strictly increasing** subsequence, we only need:

$$nums[j] < nums[i]$$

It doesn't have to be exactly one smaller.

For example:

`[2, 5, 7]`

When we're at `7`, we can extend from `2` **or** `5`.

So we consider **every** `j < i` where:

`nums[j] < nums[i]`

and choose the best one.

Think about it this way

For `nums[i]`, ask:

"What is the best increasing subsequence that I can extend with `nums[i]` ?"

For every valid previous `j`:

```
LIS ending at j
+
nums[i]
```

So your recurrence needs a **maximum over all valid j**, rather than looking for one particular value.

Try correcting **only the recurrence and condition** yourself:

```
for every j < i:
    if _____:
        dp[i] = _____
```

What should the blanks be?

12/09/2026, 17:05:34



$$\text{nums}[j] < \text{nums}[i] \quad , \quad \text{dp}[i] = \max(\text{dp}[i] , \text{dp}[j] + 1)$$

12/09/2026, 17:08:06



Exactly.

For LIS:

$$\text{dp}[i] = \max(\text{dp}[i], \text{dp}[j] + 1)$$

for every:

$$j < i \quad \text{and} \quad \text{nums}[j] < \text{nums}[i]$$

and if no such j exists:

$$\text{dp}[i] = 1$$

One subtle but important point: **the final answer is not necessarily** $\text{dp}[n-1]$, because the LIS may end before the last element.

$$\text{answer} = \max_i \text{dp}[i]$$

You've now seen the progression:

- Climbing Stairs → previous fixed states
- House Robber → choose between two states
- Decode Ways → conditional transitions
- Coin Change → any $i-c$
- LIS → any $j < i$ satisfying a condition

Let's continue.

Problem 6 — Word Break

Given a string s and a dictionary of words wordDict , determine whether s can be segmented into a sequence of dictionary words.

Each dictionary word can be used multiple times.

Example:

```
s = "leetcode"
wordDict = ["leet", "code"]

Output = true
```

Because:

```
"leet" + "code"
```

Another:

```
s = "applepenapple"
wordDict = ["apple", "pen"]

Output = true
```

Derive the DP

1. What does `dp[i]` represent?
2. What is the **last action** when forming the first `i` characters?
3. If the last word has length `k`, what previous state do you reach?
4. What conditions must be satisfied for that transition?
5. What is the recurrence?
6. What is the base case?

No code, just derive the recurrence.

12/09/2026, 17:08:07



donno about this one , point me in the right direction

12/09/2026, 17:41:37



No problem. Let's **not jump to the recurrence yet**. I'll guide you toward it.

Take:

```
s = "leetcode"
wordDict = ["leet", "code"]
```

Think of the string as positions:

```
l e e t c o d e
0 1 2 3 4 5 6 7
```

Suppose we're trying to determine whether the **first 4 characters** can be segmented:

```
"leet"
```

They can, because `"leet"` is in the dictionary.

Now suppose we're at the first 8 characters:

```
"leetcode"
```

The key question is:

What could the LAST dictionary word be?

It could be `"code"`.

If `"code"` is the last word, what remains before it?

```
"leet" | "code"
      ↑
```

previous part

So we'd need to know whether the **prefix before the last word** was already segmentable.

Now generalize

Let:

$dp[i]$ = whether the **first** i **characters** can be segmented into dictionary words.

Imagine you're calculating $dp[i]$.

Ask:

"If the last word has length k , where did that word start?"

It starts at:

$i - k$

So the string looks like:

[first $i-k$ characters]	[last word]
$dp[i-k]$	?

For example:

leetcode	
←-----→	←-----→
leet	code
$dp[4]$	word

Now the crucial question:

When can we say $dp[i] = \text{true}$?

Think about these two things:

1. Was the prefix before the last word segmentable?

2. Is the last piece actually present in `wordDict` ?

Try filling this:

`dp[i] = true` if there exists some `k` such that:

AND

That's the direction you need. Don't derive the whole recurrence yet—just fill those two conditions.

12/09/2026, 17:41:43



```
dp[i] = dp[i] | (d[i - k] and dp[i - k] and input_word[i - k : i] == word for word in dict
```

12/09/2026, 17:47:47



You're **very close**. The core idea is right: choose a dictionary word as the final piece and check whether the prefix before it is already breakable.

The clean way to express what you were trying to write is:

$$dp[i] = \text{true}$$

if there exists a word `word` such that:

$$dp[i - |word|] = \text{true}$$

and

$$s[i - |word| : i] = \text{word}$$

In other words:

[first i-k characters]	[last k characters]
dp[i-k]	word
✓	✓

For `"leetcode"`:

```
i = 8
word = "code"
k = 4

s[0:4] = "leet"    → dp[4] = true
s[4:8] = "code"    → dictionary word

therefore dp[8] = true
```

One thing to fix in your notation

You wrote:

```
d[i-k]
```

You don't need that. The dictionary check is simply:

```
s[i-k:i] in wordDict
```

So the conceptual recurrence is:

$$dp[i] = \exists word \in wordDict : dp[i - |word|] \wedge s[i - |word| : i] = word$$

with:

$$dp[0] = true$$

The important new pattern here is:

Prefix DP + try every possible final word.

That completes the main Prefix DP progression from the material.

12/09/2026, 17:47:49

[next](#)

12/09/2026, 17:48:14



Let's move to the next pattern from the material: **Two-Sequence DP**.

Problem 7 — Longest Common Subsequence (LCS)

Given two strings `text1` and `text2`, find the length of their **longest common subsequence**.

A subsequence does not need to be contiguous, but the relative order must be preserved.

Example:

```
text1 = "abcde"
```

```
text2 = "ace"
```

```
Output = 3
```

One LCS is:

```
a → c → e
```

Derive the DP

1. What does `dp[i][j]` represent?
2. We're considering the `i`-th character of `text1` and `j`-th character of `text2`. What are the two possibilities?
3. If the two characters **match**, what previous state should you come from?
4. If they **don't match**, what choices do you have?
5. What is the recurrence?
6. What are the base cases?

Think in terms of:

"What happens to the last characters of the two sequences?"

No code yet.

12/09/2026, 17:48:17



1. lcs of text1[0...i - 1] and text2[0...j - 1] 2. match or not 3. dp[i - 1][j - 1] 4. dp[i - 1][j] or dp[i][j - 1] (ie leaving one of the characters) 5. dp[i] = dp[i - 1][j - 1] + 1 if match else max(dp[i - 1][j] , dp[i][j - 1]) 6. dp[0][0] = 0

12/09/2026, 17:52:12



Exactly right. Your LCS derivation is essentially correct. Just **two small fixes**.

1. State

Your definition:

$dp[i][j] = \text{LCS of } text1[0 \dots i-1] \text{ and } text2[0 \dots j-1]$

Perfect.

We use $i-1$ and $j-1$ because $dp[i][j]$ represents the **first i and first j characters**, so their actual indices are $i-1$ and $j-1$.

2. Two possibilities

Correct:

- Characters match
- Characters don't match

Let:

text1[i-1]
text2[j-1]

be the current characters.

3. If they match

You said:

```
dp[i-1][j-1]
```

That's the previous LCS, but the **full result** is:

$$dp[i][j] = dp[i-1][j-1] + 1$$

Why?

If:

```
text1[i-1] == text2[j-1]
```

we can include this character in the LCS.

```
current matching chars
      ↓
text1: ... A
text2: ... A

      ↓ remove both

text1: ...
text2: ...

      ↓ solve

dp[i-1][j-1]

      ↓ add the matching A

+ 1
```

4. If they don't match

Exactly:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

You're essentially saying:

"If these two characters don't match, one of them cannot be part of this particular matching."

So try:

Remove `text1[i-1]` → `dp[i-1][j]`

OR

Remove `text2[j-1]` → `dp[i][j-1]`

and take the better result.

5. Recurrence - almost perfect

You wrote:

`dp[i] = ...`

Since this is a **2D state**, it should be:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } text1[i-1] = text2[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{otherwise} \end{cases}$$

This is the key Two-Sequence DP pattern.

6. Base case - one small correction

You said:

`dp[0][0] = 0`

That's true, but we need **the entire first row and first column**:

$$dp[0][j] = 0$$

$$dp[i][0] = 0$$

Because if either sequence is empty, there is no common subsequence.

For example:

""	""	a	c	e
""	0	0	0	0
a	0			
b	0			
c	0			
d	0			
e	0			

The important DP thinking

Notice what you just did.

You didn't memorize the LCS recurrence. You derived it from:

"What happens to the last characters of the two sequences?"

That's exactly how the material wants you to approach Two-Sequence DP. [dp-pattern-cheat-sheet.pdf](#)

The pattern is:

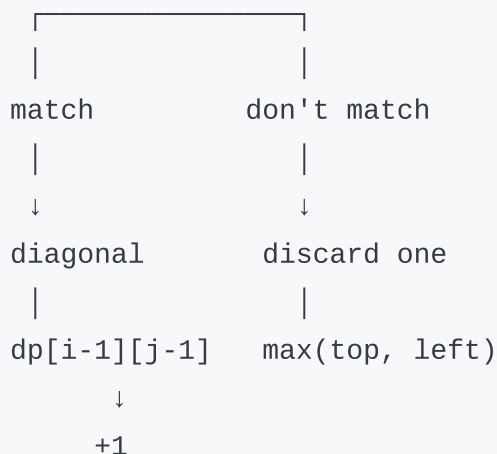
Two sequences



$dp[i][j]$



Look at $A[i-1]$, $B[j-1]$



One thing to lock in

For **Prefix DP**, you were asking:

"What was the last action?"

For **Two-Sequence DP**, your new question is:

"What happens to the last elements of the two sequences?"

That's the mental trigger.

Next we'll do **Edit Distance**, which looks very similar to LCS but introduces **three choices** instead of two.

12/09/2026, 17:52:14

[next](#)

12/09/2026, 17:52:47



Thinking

- **Located the next edit distance problem**

✓ Worked for a couple of seconds

Problem 8 - Edit Distance

This is the **next Two-Sequence DP pattern** in your material. The material specifically groups Edit Distance with LCS under Two-Sequence DP and says to ask:

“What happens to the last characters?” `dp-pattern-cheat-sheet.pdf`

Problem

Given two strings:

```
word1 = "horse"  
word2 = "ros"
```

Find the **minimum number of operations** needed to convert `word1` into `word2`.

Allowed operations:

1. **Insert** a character
2. **Delete** a character
3. **Replace** a character

For example:

```
horse  
↓ delete h  
orse  
↓ replace e → s
```

```
ors
  ↓ delete r
os
```

That's not necessarily optimal; the DP finds the minimum.

Step 1 - Identify the pattern

We have:

```
word1
word2
```

Two sequences → **Two-Sequence DP**

So:

$$dp[i][j]$$

should represent something about the first `i` characters of `word1` and first `j` characters of `word2`. `dp-pattern-cheat-sheet.pdf`

Your turn

Derive it exactly like you did for LCS.

Answer these **5 things**:

1. What does `dp[i][j]` represent?
2. What happens if `word1[i-1] == word2[j-1]`?
3. If they don't match, what **three operations** can we perform?
4. What previous DP state corresponds to each operation?
5. What should the base cases `dp[0][j]` and `dp[i][0]` be?

Don't worry about the recurrence yet-derive those first.

12/09/2026, 17:52:51



1. min operations to convert word1[0..i-1] into word2[0..j-1] 2. no operations 3. insertion , deletion , replace 4. insertion $dp[i - 1][j] + 1$, deletion $dp[i][j - 1] + 1$, replace $dp[i - 1][j - 1] + 1$ 5. $dp[i][0] = i$
 $dp[0][j] = j$

12/09/2026, 17:59:03



Yes - **almost completely correct**. There is just one important issue: you swapped the names of **insert** and **delete** relative to the DP states.

1. State

$dp[i][j]$ = minimum operations to convert $word1[:i] \rightarrow word2[:j]$

Exactly right.

2. Characters match

If:

$$word1[i - 1] = word2[j - 1]$$

then we don't need an operation:

$$dp[i][j] = dp[i - 1][j - 1]$$

3. Three operations

Correct:

- Insert
- Delete
- Replace

4. The small correction

According to the material:

Operation	Previous state
Delete current char from <code>word1</code>	<code>dp[i-1][j] + 1</code>
Insert required char into <code>word1</code>	<code>dp[i][j-1] + 1</code>
Replace current char	<code>dp[i-1][j-1] + 1</code>

You had insertion/deletion reversed. The states themselves were correct. `dp-pattern-cheat-sheet.pdf`

Think about it:

```
dp[i-1][j]
```

means we've removed one character from `word1` → **delete**.

```
dp[i][j-1]
```

means we've effectively supplied one more character toward `word2` → **insert**.

Therefore:

$$dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$$

when the characters don't match. `dp-pattern-cheat-sheet.pdf`

5. Base cases

Perfect:

$$dp[i][0] = i$$

To convert `"abc"` → `""`, delete 3 characters.

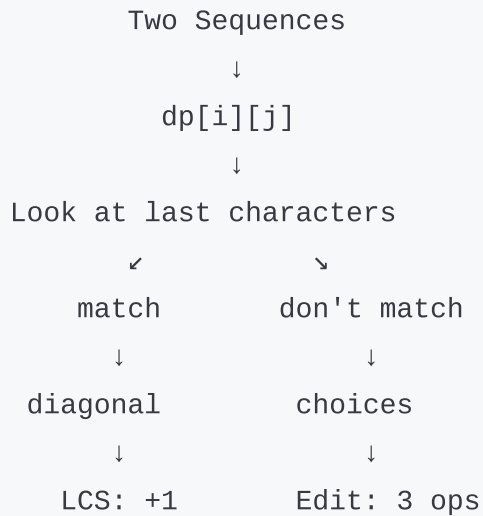
And:

$$dp[0][j] = j$$

To convert `""` → `"abc"`, insert 3 characters. `dp-pattern-cheat-sheet.pdf`

🔑 What you should remember

LCS and Edit Distance have the **same DP shape**:



The material's key recognition rule is essentially:

Two sequences → `dp[i][j]` → **ask what happens to the last characters.** `dp-pattern-cheat-sheet.pdf`

Next: Distinct Subsequences

This is another Two-Sequence DP problem, but this time the DP stores a **count** rather than a minimum/maximum.

I'll make you derive it again rather than giving the recurrence.

12/09/2026, 17:59:06



one thing i don't understand with respect to what you are classifying this insertion , del ,replace explain clearly

12/09/2026, 18:01:02



Yes - the confusing part is **“insertion/deletion/replacement with respect to what?”**

In **Edit Distance**, we are always asking:

How do I transform `word1` **into** `word2` ?

So insertion/deletion/replacement are classified based on **what operation we perform on** `word1` **to make it become** `word2` .

Example

Suppose:

`word1 = "cat"`

`word2 = "cut"`

We want:

`cat → cut`

The characters:

```
c a t
  ↓
c u t
```

We can **replace** `a` with `u` .

So that's a **replacement**.

Now consider `cat → cats`

```
cat → cats
    ↑
    s
```

We need to **add** `s` to `word1`.

That's an **insertion**.

```
cats → cat
```

```
cats → cat
    ↑
  remove s
```

We need to **remove** `s` from `word1`.

That's a **deletion**.

Now connect this to `dp[i][j]`

The material defines:

`dp[i][j]` = minimum operations needed to convert the first `i` characters of `word1` into the first `j` characters of `word2` . [dp-pattern-cheat-sheet.pdf](#)

Suppose:

```
word1 = "abc"
word2 = "adc"
```

At:

```
dp[2][2]
```

we are converting:

```
word1[:2] = "ab"
    ↓
word2[:2] = "ad"
```

The last characters are:

```
a b
  ↓
a d
```

`b != d`, so we have three possible things we could do:

1. Delete `b` from word1

```
ab → a
```

Then we still need to convert `a → ad`.

So:

```
dp[i-1][j] + 1
```

2. Insert `d` into word1

```
ab → abd
```

Conceptually, we've added the required `d`.

So:

```
dp[i][j-1] + 1
```

3. Replace `b` with `d`

```
ab → ad
```

So:

```
dp[i-1][j-1] + 1
```

The important mental model

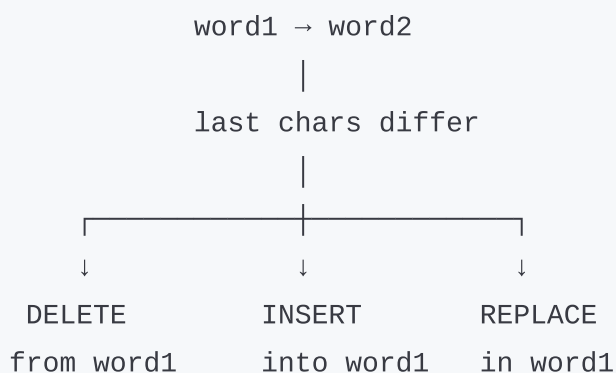
Don't think:

"Insertion means something related to i , deletion means something related to j ."

Instead think:

I'm transforming word1 → word2. What can I do to the current characters of word1 to eventually produce word2?

That's why the three choices are:



And **this classification is specifically for the operation we perform on word1**.

One subtle point: the DP recurrence's $i-1$ / $j-1$ indices can make insertion/deletion look reversed at first. The safest way is to always ask **“what operation am I performing on word1?”** Then the recurrence follows.

12/09/2026, 18:01:04



ok , next

12/09/2026, 18:03:40



Good. **Next: Distinct Subsequences** - this is the next Two-Sequence DP problem in your material. The material classifies it as Two-Sequence DP because we have two strings and `dp[i][j]` represents the first `i` characters of one and first `j` characters of the other. `dp-pattern-cheat-sheet.pdf`

3. Distinct Subsequences

Problem:

Given strings `s` and `t`, find **how many distinct subsequences of `s` are equal to `t`**.

Example:

```
s = "rabbbit"
t = "rabbit"
```

Answer:

3

Why 3? There are three different ways to choose the three `b` s from `"rabbbit"` to form `"rabbit"`.

Step 1 - Classify it

We have:

```
s = sequence 1
t = sequence 2
```

So:

```
dp[i][j]
```

Now I want **you to derive what it means.**

Think about:

```
s[:i]  
t[:j]
```

What should `dp[i][j]` represent?

And then we'll ask the same question we've been using:

What happens to the last characters?

Give me your definition of `dp[i][j]` first.

12/09/2026, 18:03:43



distinct subsequences of $s[:i]$ matching $t[:j]$

12/09/2026, 18:12:18



Exactly. ✓

So:

$dp[i][j]$ = number of distinct subsequences of $s[:i]$ that equal $t[:j]$.

Now the key question.

Suppose:

```
s[:i] = "...a"
t[:j] = "...a"
```

The last characters **match**.

Unlike LCS, here we are **counting ways**.

What can we do with the last **a** of **s**?

Think carefully — there are **two possibilities**:

1. Use this **a** to match the last **a** of **t**
2. Don't use this **a**; skip it and try to form $t[:j]$ from the earlier part of **s**

So what do you think the recurrence should be for the **matching case**?

Try writing:

```
dp[i][j] = ?
```

Then we'll handle the mismatch case.

12/09/2026, 18:12:22



1. option1 donno

12/09/2026, 18:35:39



Yes — **option 1 is correct.**

For the matching case:

$$dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$$

Why?

- `dp[i-1][j-1]` → **use** `s[i-1]` to match `t[j-1]`
- `dp[i-1][j]` → **skip** `s[i-1]`

So unlike LCS, where matching characters essentially gives one path, here **both possibilities count.**

Now the important part: mismatch

Suppose:

```
s[i-1] != t[j-1]
```

Can we use `s[i-1]` to match `t[j-1]` ?

No. Therefore, what should we do with `s[i-1]` ?

Try to derive:

$$dp[i][j] = ?$$

Just give me the expression.

12/09/2026, 18:35:42



either use $s[i - 1]$ to match the previous $t[:j - 1]$ else ignore $s[i - 1]$ so, $dp[i][j] = dp[i][j - 1] + dp[i - 1][j - 1]$
 i have a doubt, for matching case, cant we just keep $dp[i - 1][j - 1]$ alone?

12/09/2026, 18:39:09



Yes — this is exactly the subtle point in **Distinct Subsequences**.

Your recurrence for the mismatch case is **not correct**, and your doubt about the matching case is important.

Matching case

Suppose:

```
s[i-1] == t[j-1]
```

For example:

```
s = "bab"
t = "ba"
```

At the final **b**, we have **two choices**:

Choice 1 — use this **b**

Use $s[i-1]$ to match $t[j-1]$.

Then we need to form $t[:j-1]$ from $s[:i-1]$:

$$dp[i - 1][j - 1]$$

Choice 2 — don't use this **b**

We can **skip this** **b** and see whether the earlier characters of **s** can still form the entire **t**:

$$dp[i - 1][j]$$

Therefore:

$$dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$$

Why can't we keep only `dp[i-1][j-1]` ?

Because that would mean:

*"Whenever the characters match, I **must** use this character."*

But subsequences don't work that way. A character can be matching **and still be skipped**.

For example:

```
s = "aaa"
t = "aa"
```

There are **3** distinct ways:

```
a1 a2
a1 a3
a2 a3
```

If we always used the matching last `a`, we'd miss some of these possibilities.

That's why we need **both branches**.

Now your mismatch case

You said:

either use `s[i-1]` to match previous `t[:j-1]` else ignore `s[i-1]`

The first option is impossible when:

```
s[i-1] != t[j-1]
```

You cannot use `s[i-1]` to match `t[j-1]`, because they are different.

So there is **only one choice**:

Skip $s[i-1]$.

Therefore:

$$dp[i][j] = dp[i-1][j]$$

So the complete recurrence is:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + dp[i-1][j], & s[i-1] = t[j-1] \\ dp[i-1][j], & s[i-1] \neq t[j-1] \end{cases}$$

The key idea to remember:

Match → USE or SKIP

Mismatch → SKIP only

12/09/2026, 18:39:13



ok , next

12/09/2026, 18:42:27



Thinking

- **Identified the next dynamic programming topic**

✓ Worked for a few seconds

Great. That completes the **Two-Sequence DP** section covered so far: LCS, Edit Distance, and Distinct Subsequences. The material next moves to **Interval DP**. `dp-pattern-cheat-sheet.pdf`

Interval DP

The central idea is very simple:

Instead of asking about a prefix, ask about a range `[i, j]`.

The state becomes:

$$dp[i][j] = \text{answer for the interval } [i, j]$$

The material says to recognize Interval DP when the problem involves things like:

- substring
- subarray
- segment
- range
- interval

Examples include **Longest Palindromic Subsequence, Burst Balloons, Matrix Chain Multiplication, and Palindrome Partitioning**. `dp-pattern-cheat-sheet.pdf`

The important mental question

For Interval DP, don't immediately think about the recurrence.

Think:

"What happens to the two ends of this interval?"

For example, suppose:

```
a  b  c  d  a
↑           ↑
i           j
```

Your subproblem is:

```
dp[i][j]
```

meaning:

What is the answer inside `i ... j`?

Then you examine the **left end** `i` and **right end** `j`.

For example, in Longest Palindromic Subsequence:

```
a  . . . .  a
↑           ↑
i           j
```

If the ends match:

$$dp[i][j] = 2 + dp[i + 1][j - 1]$$

If they don't:

$$dp[i][j] = \max(dp[i + 1][j], dp[i][j - 1])$$

This is the basic **shrinking interval** style of Interval DP. [dp-pattern-cheat-sheet.pdf](#)

But there's another very important style inside Interval DP:

Partition DP

Instead of asking:

"What happens to the two ends?"

you ask:

"Where should I split this interval?"

You try every possible split point k :

```
[i ..... j]
    ↑
    k

[i ... k] + [k+1 ... j]
```

So the recurrence generally looks like:

$$dp[i][j] = \min / \max_k (dp[i][k] + dp[k+1][j] + cost)$$

The material specifically treats **Partition DP as a subtype of Interval DP**. [dp-pattern-cheat-sheet.pdf](#)

So remember the distinction

Type	Main question
Prefix DP	What happens up to i ?
Two-Sequence DP	What happens with the two sequences?
Shrinking Interval DP	What happens to the ends i, j ?
Partition DP	Where should I split $i \dots j$?

And both use:

$$dp[i][j]$$

The **meaning of `i` and `j`** is what tells you which one you're dealing with.

Let's learn **Shrinking Interval DP first**, using **Longest Palindromic Subsequence (LPS)**.

Define `dp[i][j]` for LPS in your own words.

12/09/2026, 18:42:31



you forgot one thing , give the template codes for memo , table , for both two sequence and interval dp (shrinking , partition both)

12/09/2026, 19:04:00



You're right - **I should include the reusable code templates** as we go. We'll keep them as part of each pattern.

Here are the four templates you should memorize/understand.

1. Two-Sequence DP - Memoization

Used when:

```
dp[i][j]
i → position/prefix in sequence 1
j → position/prefix in sequence 2
```

Generic template:

```
def solve(i, j):
    # Base case
    if i == 0 or j == 0:
        return base_case

    if (i, j) in memo:
        return memo[(i, j)]

    if condition: # e.g. s[i-1] == t[j-1]
        ans = ...
    else:
        ans = ...

    memo[(i, j)] = ans
    return ans
```

```
memo = {}  
answer = solve(n, m)
```

For Python, you can also use a 2D array:

```
memo = [[-1] * (m + 1) for _ in range(n + 1)]
```

2. Two-Sequence DP - Tabulation

Start from the base cases and build toward (n, m) .

```
dp = [[0] * (m + 1) for _ in range(n + 1)]  
  
# Base cases  
for i in range(...):  
    dp[i][0] = ...  
  
for j in range(...):  
    dp[0][j] = ...  
  
# Fill table  
for i in range(1, n + 1):  
    for j in range(1, m + 1):  
  
        if condition:  
            dp[i][j] = ...  
        else:  
            dp[i][j] = ...  
  
return dp[n][m]
```

For **LCS**, for example:

```
if text1[i - 1] == text2[j - 1]:  
    dp[i][j] = 1 + dp[i - 1][j - 1]  
else:  
    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
```

3. Interval DP - Shrinking Interval

Here:

```
dp[i][j] = answer for interval [i, j]
```

Memoization

```
def solve(i, j):

    if i > j:
        return base_case

    if i == j:
        return base_case

    if (i, j) in memo:
        return memo[(i, j)]

    if condition:          # e.g. nums[i] == nums[j]
        ans = ...
    else:
        ans = ...

    memo[(i, j)] = ans
    return ans

memo = {}
answer = solve(0, n - 1)
```

The important thing is that the interval **shrinks**:

[i j]

↓ ↓

[i+1 ... j-1]

or perhaps:

```
[i ..... j]
↓
[i+1 ..... j]
```

or:

```
[i ..... j]
↓
[i ..... j-1]
```

depending on the problem.

Tabulation

For shrinking interval DP, the crucial thing is the **order**.

You generally solve **smaller intervals first**:

```
dp = [[0] * n for _ in range(n)]

# Base cases
for i in range(n):
    dp[i][i] = ...

# Increasing interval length
for length in range(2, n + 1):

    for i in range(n - length + 1):

        j = i + length - 1

        if condition:
            dp[i][j] = ...
        else:
            dp[i][j] = ...

return dp[0][n - 1]
```

This is because something like:

```
dp[i + 1][j - 1]
```

must already have been calculated.

4. Interval DP - Partition

This one is extremely important.

State:

```
dp[i][j] = answer for interval [i, j]
```

But now we ask:

Where should I split `[i, j]` ?

```
[i ..... j]
    ↑
    k

[i ..... k] [k+1 ..... j]
```

Memoization

```
def solve(i, j):

    if i >= j:
        return base_case

    if (i, j) in memo:
        return memo[(i, j)]

    ans = INF

    for k in range(i, j):

        left = solve(i, k)
        right = solve(k + 1, j)
```

```

        cost = ...

        ans = min(ans, left + right + cost)

    memo[(i, j)] = ans
    return ans

memo = {}
answer = solve(0, n - 1)

```

For a **maximum** problem:

```

ans = -INF
ans = max(ans, left + right + cost)

```

For a **counting** problem, the structure changes accordingly.

Tabulation

Partition DP has a very recognizable template:

```

dp = [[0] * n for _ in range(n)]

for length in range(2, n + 1):

    for i in range(n - length + 1):

        j = i + length - 1

        dp[i][j] = INF

        for k in range(i, j):

            left = dp[i][k]
            right = dp[k + 1][j]

            cost = ...

            dp[i][j] = min(

```

```

        dp[i][j],
        left + right + cost
    )

return dp[0][n - 1]

```

This `length → i → j → k` structure is **the big template to recognize**.

The four templates side-by-side

Pattern	Memo	Tabulation
Two Sequence	<code>solve(i, j)</code>	<code>i → j</code>
Shrinking Interval	<code>solve(i, j)</code>	<code>length → i → j</code>
Partition Interval	<code>solve(i, j) + k</code> loop	<code>length → i → j → k</code>

And the most important distinction:

Two Sequence:

`i` → sequence 1

`j` → sequence 2

Shrinking Interval:

`i` → LEFT boundary

`j` → RIGHT boundary

```

[ i .... j ]
  ↘     ↗
  shrink

```

Partition:

`i` → LEFT boundary

`j` → RIGHT boundary

`k` → SPLIT POINT

```

[ i .... k | k+1 .... j ]

```

↑
split

This matches the material's classification: two indices belonging to different sequences indicate Two-Sequence DP, while two indices representing the same range indicate Interval DP; Partition DP then introduces a split point `k`. [dp-pattern-cheat-sheet.pdf](#)

We'll use these templates from now on while solving the problems, rather than only deriving the recurrence.

Now let's continue with **Longest Palindromic Subsequence (LPS)** and derive its state → choices → recurrence → base cases → **memo code** → **table code**.

12/09/2026, 19:04:02



ok , continue with the problems

12/09/2026, 19:09:40



Yes. We'll continue exactly from the material, and now move into **Interval DP** → **Shrinking Interval DP**, starting with **Longest Palindromic Subsequence (LPS)**.

The material classifies LPS as Interval DP because the state represents an interval `[i, j]`, and the key question is **“What happens to the ends?”** [dp-pattern-cheat-sheet.pdf](#)

Problem 1 - Longest Palindromic Subsequence

Given a string `s`, find the length of the longest subsequence of `s` that is a palindrome.

Example:

```
s = "bbbab"
```

```
Longest palindromic subsequence = "bbbb"
```

```
answer = 4
```

Remember:

- **Substring** → contiguous
- **Subsequence** → can skip characters

So for:

```
a b c d a
```

`aba` can be a subsequence, even though it isn't a substring. The material specifically uses this distinction when explaining LPS. [dp-pattern-cheat-sheet.pdf](#)

Step 1 - Classify it

Ask:

What kind of subproblem am I dealing with?

We're working with a **range of the same string**:

```
s[i ... j]
```

Therefore:

Interval DP

More specifically:

```
Interval DP
  ↓
Shrinking Interval DP
  ↓
Longest Palindromic Subsequence
```

Unlike Partition DP, we **don't try every split** `k`. We reason about the two ends. `dp-pattern-cheat-sheet.pdf`

Step 2 - Define the state

What should:

```
dp[i][j]
```

mean?

Try to formulate it yourself:

```
dp[i][j] = ?
```

Think:

What is the answer for the interval `s[i...j]`?

Don't worry about the recurrence yet.

12/09/2026, 19:09:43



longest palindromic subsequence of $s[i...j]$

12/09/2026, 19:13:22



Exactly. ✓

So our state is:

```
dp[i][j] = length of the longest palindromic subsequence
           inside s[i...j]
```

This is the **correct state** and matches the material. `dp-pattern-cheat-sheet.pdf`

Now comes the most important part.

Step 3 - What happens to the two ends?

Look at:

```
s[i] ..... s[j]
  ↑           ↑
left        right
```

A palindrome reads the same forwards and backwards.

So there are two major cases.

Case 1: `s[i] == s[j]`

Example:

```
b ... b
```

If the two ends match, we can potentially **use both of them** in our palindrome.

So we get:

```
2 + dp[i+1][j-1]
```

Why?

```
b [middle] b
↑           ↑
2 characters
```

The middle must give us the best palindromic subsequence, which is:

```
dp[i+1][j-1]
```

Therefore:

```
dp[i][j] = 2 + dp[i+1][j-1]
```

Case 2: `s[i] != s[j]`

Example:

```
b ... a
```

We cannot use **both ends together** as the two matching ends of the palindrome.

But the answer might still contain either:

```
b ...
```

or:

```
... a
```

So we have two choices:

Skip the left character:

```
dp[i+1][j]
```

Skip the right character:

```
dp[i][j-1]
```

Take the better one:

```
dp[i][j] = max(  
    dp[i+1][j],  
    dp[i][j-1]  
)
```

This is exactly the material's shrinking-interval thought process: **matching ends → take both; otherwise → skip left or right.** [dp-pattern-cheat-sheet.pdf](#)

So derive the full recurrence

```
if s[i] == s[j]:  
    dp[i][j] = 2 + dp[i+1][j-1]  
  
else:  
    dp[i][j] = max(  
        dp[i+1][j],  
        dp[i][j-1]  
    )
```

Now **you derive the base cases.**

Think about the smallest possible intervals:

1. What is the LPS of a single character?
2. What should happen when `i > j` (an empty interval)?

Give me those two base cases.

12/09/2026, 19:13:24



1.12.0

12/09/2026, 19:14:04



Exactly.

For **Longest Palindromic Subsequence (LPS)**:

- `i == j` → one character → `dp[i][j] = 1`
- `i > j` → empty interval → `dp[i][j] = 0`

So our complete recurrence is:

$$dp[i][j] = \begin{cases} 2 + dp[i+1][j-1] & s[i] = s[j] \\ \max(dp[i+1][j], dp[i][j-1]) & s[i] \neq s[j] \end{cases}$$

Now let's implement it

Since this is **Shrinking Interval DP**, the material's memoization template becomes:

```
def solve(i, j):
    if i > j:
        return 0
    if i == j:
        return 1

    if (i, j) in memo:
        return memo[(i, j)]

    if s[i] == s[j]:
        ans = 2 + solve(i + 1, j - 1)
    else:
        ans = max(
            solve(i + 1, j),
            solve(i, j - 1)
        )

    memo[(i, j)] = ans
```

```
return ans
```

```
memo = {}  
answer = solve(0, n - 1)
```

The important thing is **not the syntax**. Notice the structure:

```
solve(i, j)  
  |  
  ├── empty? → 0  
  ├── one char? → 1  
  |  
  ├── s[i] == s[j]?  
  |       └─ take BOTH ends  
  |  
  └─ otherwise  
      ├── skip left  
      └─ skip right
```

This is exactly why it's called **shrinking interval**: every recursive call makes `[i, j]` smaller.

One important question

For the string:

```
"bbbab"
```

What is the LPS?

Don't calculate using the DP table yet. **Try to identify the subsequence itself** and give me its length.

12/09/2026, 19:14:06



leave it move to next question , also , dont give the answer

12/09/2026, 19:14:42



Got it — we'll skip solving that example, and **I won't give answers to the problems unless you ask for the answer.**

Next: Partition DP

Let's move to the next problem from the material: **Matrix Chain Multiplication (MCM).**

First question only:

How would you classify Matrix Chain Multiplication?

- A. Prefix DP
- B. Two-Sequence DP
- C. Interval DP — Shrinking
- D. Interval DP — Partition
- E. Knapsack DP

12/09/2026, 19:14:44



first state the problem , i dont know matrix chain multiplication

12/09/2026, 19:15:10



Absolutely. Let's first understand the problem itself—**no DP yet**.

Matrix Chain Multiplication

Suppose you have three matrices:

$$A \times B \times C$$

with dimensions:

$$A = 10 \times 20$$

$$B = 20 \times 30$$

$$C = 30 \times 40$$

You need to multiply all three.

The important thing is that **matrix multiplication is associative**:

$$(A \times B) \times C$$

and

$$A \times (B \times C)$$

produce the same final matrix.

But they can require **very different numbers of scalar multiplications**.

Option 1

$$(A \times B) \times C$$

First:

$$A (10 \times 20) \times B (20 \times 30)$$

Cost:

$$10 \times 20 \times 30 = 6000$$

The result is 10×30 .

Then:

$$(10 \times 30) \times C (30 \times 40)$$

Cost:

$$10 \times 30 \times 40 = 12000$$

Total:

$$6000 + 12000 = 18000$$

Option 2

$$A \times (B \times C)$$

First:

$$B (20 \times 30) \times C (30 \times 40)$$

Cost:

$$20 \times 30 \times 40 = 24000$$

Then:

$$A (10 \times 20) \times (20 \times 40)$$

Cost:

$$10 \times 20 \times 40 = 8000$$

Total:

$$24000 + 8000 = 32000$$

So:

$$(A \times B) \times C \rightarrow 18,000 \text{ operations}$$

$$A \times (B \times C) \rightarrow 32,000 \text{ operations}$$

Same final result, different computational cost.

The actual problem

Given a sequence of matrices:

$$A_1 \times A_2 \times A_3 \times \dots \times A_n$$

find the parenthesization that minimizes the number of scalar multiplications.

We're **not actually multiplying the matrices**. We're finding the **best order to perform the multiplications**.

Now that the problem is clear, we'll derive the DP ourselves.

Question 1: What kind of DP does this look like based on the shape of the problem?

- A. Prefix
- B. Two-Sequence
- C. Interval — Shrinking
- D. Interval — Partition
- E. Knapsack

12/09/2026, 19:15:16



D

12/09/2026, 19:43:35



Correct.  **D — Interval DP → Partition.**

Now let's derive the **state**.

For MCM, we have a chain:

$A_1 \times A_2 \times A_3 \times \dots \times A_n$

The key question is:

If I focus on a continuous portion of the chain, what should my DP state represent?

For example:

$A_1 \times A_2 \times A_3 \times A_4$

└──────────┘

What would you define as `dp[i][j]` ?

Just give me the state definition in words.

12/09/2026, 19:43:37



donno

12/09/2026, 19:44:12



No problem. Let's make the **state** intuitive before asking you again.

We have:

$A1 \times A2 \times A3 \times A4$

In interval DP, **i** and **j** represent the **boundaries of the part we're currently solving**.

So:

$dp[i][j]$

should mean:

*"What is the minimum multiplication cost for multiplying matrices from **Ai through Aj?**"*

For example:

$dp[1][3]$

means:

$A1 \times A2 \times A3$

and asks for the **minimum cost to multiply those three matrices**.

So now try to state it yourself:

$dp[i][j] = ?$

Just one sentence.

12/09/2026, 19:44:14



minimum matrix multiplication cost for multiplything matrix i through j

12/09/2026, 19:44:55



Exactly. ✓

$dp[i][j]$ = *minimum multiplication cost for multiplying matrices A_i through A_j .*

Now the key part of **Partition DP**:

Suppose we want to solve:

$A_1 \times A_2 \times A_3 \times A_4$

We need to decide **where to split the chain**.

For example:

$(A_1) \times (A_2 \times A_3 \times A_4)$

or

$(A_1 \times A_2) \times (A_3 \times A_4)$

or

$(A_1 \times A_2 \times A_3) \times (A_4)$

So we introduce a split point k .

For $dp[i][j]$, k can be anywhere between i and $j-1$:

$A_i \dots A_k \mid A_{(k+1)} \dots A_j$
 $\uparrow$
 k

Your turn

If we're calculating `dp[1][4]` , what are **all possible values of** `k` ?

Just give the values.

12/09/2026, 19:44:57



1, 2, 3?

12/09/2026, 20:39:34



Yes, exactly. ✓

For:

$A_1 \times A_2 \times A_3 \times A_4$

the possible split points are:

$k = 1, 2, 3$

Because the split happens between:

k=1: $A_1 \mid A_2 A_3 A_4$

k=2: $A_1 A_2 \mid A_3 A_4$

k=3: $A_1 A_2 A_3 \mid A_4$

This is the **core idea of Partition DP**: try every possible k , solve the left and right intervals, and choose the best one.

Next question

For a particular split k , we have:

$A_i \dots A_k \mid A_{(k+1)} \dots A_j$

What two **DP subproblems** would we need to solve?

Think in terms of our state $dp[i][j]$.

12/09/2026, 20:39:35



$dp[i][k]$, $dp[k + 1][j]$

12/09/2026, 20:40:29



Exactly. ✓

For a split at k :

$A_i \dots A_k \mid A_{(k+1)} \dots A_j$

we solve:

$dp[i][k]$ → minimum cost for the left part
 $dp[k+1][j]$ → minimum cost for the right part

But we're **not done**. After multiplying those two resulting matrices, there's an additional cost.

Next question

Suppose:

$A = p \times q$
 $B = q \times r$

What is the number of scalar multiplications needed to calculate $A \times B$?

Think back to the basic matrix multiplication rule.

12/09/2026, 20:40:31



$p * q * r$

12/09/2026, 20:41:25



Correct. ✓

If:

$$\begin{aligned} A &= p \times q \\ B &= q \times r \end{aligned}$$

then multiplying them costs:

$$p \times q \times r$$

So for our MCM split:

$$A_i \dots A_k \mid A_{(k+1)} \dots A_j$$

we have:

1. $dp[i][k]$ → cost of left part
2. $dp[k+1][j]$ → cost of right part
3. **cost of multiplying the two resulting matrices**

The next step is figuring out the dimensions of those two resulting matrices so we can express that third cost.

Let's use a concrete example next.

12/09/2026, 20:41:28



1. $dp[i][k] + dp[k + 1][j] + \text{cost}$
why

$p \times q \times r$ explain

12/09/2026, 20:43:41



Yes — $dp[i][k] + dp[k+1][j] + \text{cost}$ is correct. ✓

Now let's understand **why the extra cost is** $p \times q \times r$, because this is important.

Suppose we have:

$$\begin{aligned} A &= p \times q \\ B &= q \times r \end{aligned}$$

When we calculate $A \times B$, imagine the result:

$$\begin{array}{c} \begin{array}{c} B \\ q \times r \end{array} \\ \begin{array}{c} A \\ p \times q \end{array} \left[\begin{array}{c} \hline p \times r \\ \hline \end{array} \right] \end{array}$$

The resulting matrix has $p \times r$ **elements**.

For **each one element**, we calculate a dot product:

row of $A \times$ column of B

Each row has q elements, and each column has q elements:

$$a_1b_1 + a_2b_2 + \dots + a_qb_q$$

So each output element requires roughly q **multiplications**.

There are $p \times r$ **output elements**.

Therefore:

$$(p \times r) \times q$$

which gives:

$$p \times q \times r$$

So in MCM, after independently computing the left and right parts, we pay the cost of multiplying their resulting matrices.

Now let's connect it back

For:

$A_i \dots A_k \mid A_{(k+1)} \dots A_j$

we have:

```
left cost  = dp[i][k]
right cost = dp[k+1][j]
final multiplication cost = p × q × r
```

Therefore:

$$dp[i][k] + dp[k+1][j] + pqr$$

And because we're looking for the **minimum**, we try every possible k and take the minimum.

Next question: Suppose the matrices are represented by a dimension array:

```
dims = [10, 20, 30, 40]
```

What are the dimensions of A_1 , A_2 , and A_3 ?

12/09/2026, 20:43:43



10 * 20 , 20 * 30 , 30 * 40

12/09/2026, 20:44:56



Correct. ✓

So:

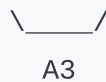
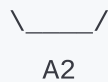
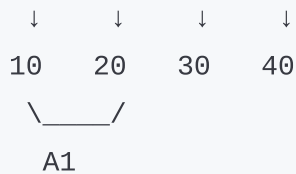
$$A1 = 10 \times 20$$

$$A2 = 20 \times 30$$

$$A3 = 30 \times 40$$

The dimension array works like this:

```
dims = [10, 20, 30, 40]
```



So generally:

`Ai` has dimensions `dims[i-1] × dims[i]`.

Now let's apply this to a split.

Suppose we're calculating `dp[1][3]` and split at `k = 2`:

$$A1 \times A2 \mid A3$$

We know:

$$A1 = 10 \times 20$$

$$A2 = 20 \times 30$$

$$A3 = 30 \times 40$$

After optimally multiplying $A1 \times A2$, the resulting matrix will have what dimensions?

Don't calculate the cost - just give me the dimensions.

12/09/2026, 20:44:58



10 * 30

12/09/2026, 20:45:26



Exactly. ✓

For the split:

$$A_1 \times A_2 \mid A_3$$

we have:

- $A_1 = 10 \times 20$
- $A_2 = 20 \times 30$

So:

$$(10 \times 20)(20 \times 30) \rightarrow 10 \times 30$$

Now let's finish this split.

The resulting matrix is 10×30 , and A_3 is 30×40 .

So the final multiplication is:

$$(10 \times 30)(30 \times 40)$$

Question: what is the scalar multiplication cost of this final multiplication?

Don't calculate the whole MCM problem—just this one cost.

12/09/2026, 20:45:27



10 * 30 * 40

12/09/2026, 20:45:40



Exactly. ✓

$$10 \times 30 \times 40 = 12,000$$

So for the split $k = 2$:

$$\underbrace{A_1 A_2}_{10 \times 30} \mid A_3$$

the **final multiplication cost is 12,000**.

But remember: **this is only the cost of the final multiplication**.

The total cost for this split is:

$$dp[1][2] + dp[3][3] + 10 \times 30 \times 40$$

And since $dp[3][3] = 0$:

$$dp[1][2] + 12,000$$

Now we can generalize this.

For $dp[i][j]$, if we split at k :

$$A_i \dots A_k \mid A_{k+1} \dots A_j$$

the total cost is:

$$dp[i][k] + dp[k+1][j] + dims[i-1] \times dims[k] \times dims[j]$$

The **important part** is that we try **every possible** k and take the minimum.

That's exactly why MCM is **Partition Interval DP**.

12/09/2026, 20:45:42



why i - 1 ?

12/09/2026, 20:46:43



Good question. The `i - 1` comes from **how we store the matrix dimensions**.

Suppose:

```
dims = [10, 20, 30, 40]
```

This represents:

$A_1 = 10 \times 20$

$A_2 = 20 \times 30$

$A_3 = 30 \times 40$

Notice the pattern:

Matrix	Rows	Columns
A1	<code>dims[0]</code>	<code>dims[1]</code>
A2	<code>dims[1]</code>	<code>dims[2]</code>
A3	<code>dims[2]</code>	<code>dims[3]</code>

So, in general:

$$A_i = \text{dims}[i - 1] \times \text{dims}[i]$$

Why?

Because **matrix numbering starts at 1**, but **array indexing starts at 0**.

For example:

$A_1 \rightarrow \text{dims}[0] \times \text{dims}[1]$

$A_2 \rightarrow \text{dims}[1] \times \text{dims}[2]$

$$A_3 \rightarrow \text{dims}[2] \times \text{dims}[3]$$

Therefore, if we're talking about A_i , its **number of rows** is $\text{dims}[i-1]$.

Now consider our split:

$$A_i \dots A_k \mid A_{k+1} \dots A_j$$

The left result has dimensions:

$$\text{dims}[i-1] \times \text{dims}[k]$$

The right result has dimensions:

$$\text{dims}[k] \times \text{dims}[j]$$

So their multiplication costs:

$$\boxed{\text{dims}[i-1] \times \text{dims}[k] \times \text{dims}[j]}$$

The $i-1$ isn't some special DP trick—it is simply because A_i **gets its row dimension from** $\text{dims}[i-1]$.

12/09/2026, 20:46:45



ok , next

12/09/2026, 20:47:17



Good. We've finished the **Matrix Chain Multiplication** example and the Partition DP idea.

According to your material, the next major pattern to learn is **Grid DP**. The cheat sheet classifies problems involving a matrix/grid and movement between cells as Grid DP. `dp-pattern-cheat-sheet.pdf`

Next: Grid DP

Before touching DP, let's understand the problem first.

Consider this grid:

```
1  3  1
1  5  1
4  2  1
```

You start at the **top-left** and want to reach the **bottom-right**.

Suppose you're only allowed to move:

```
→ Right
↓ Down
```

For example:

```
S  →  →
↓
↓
→  →  E
```

There can be multiple possible paths.

The classic Grid DP question is:

What is the minimum sum of values along a path from the top-left to the bottom-right?

The material gives **Minimum Path Sum** and **Unique Paths** as classic Grid DP examples. `dp-pattern-cheat-sheet.pdf`

We won't jump to the recurrence yet.

First question: What do you think `dp[r][c]` should represent in this problem?

12/09/2026, 20:47:19



min sum along 0 , 0 to r ,c

12/09/2026, 20:57:03



Yes.  That's exactly the right state:

$$dp[r][c] = \text{minimum path sum from } (0,0) \text{ to } (r,c)$$

Now think about the **last move** you make before reaching (r,c) .

If movement is only **right** and **down**, you can reach (r,c) from only:

$$\begin{array}{c} (r-1, c) \\ \downarrow \\ (r, c) \\ \uparrow \\ (r, c-1) \rightarrow \end{array}$$

So there are **two possible previous cells**.

Your turn

If $grid[r][c]$ is the value of the current cell, what should the recurrence be?

Try to write:

$$dp[r][c] = \boxed{?}$$

Don't worry about edge cases yet.

12/09/2026, 20:57:04



$$\min(dp[r - 1][c], dp[r][c - 1])$$

12/09/2026, 20:57:48



Correct.  Almost complete.

You forgot to add the **current cell's value**.

Since $dp[r][c]$ is the **total minimum sum up to** (r, c) , we need:

$$dp[r][c] = grid[r][c] + \min(dp[r - 1][c], dp[r][c - 1])$$

Why?

Suppose:

```

      5
      ↓
3 →  2

```

If we're calculating the 2 :

- Coming from above already costs 5
- Coming from left already costs 3
- Choose the cheaper path: $\min(5, 3) = 3$
- Then **add the current cell** 2

So:

$$dp = 3 + 2 = 5$$

This is the same DP thinking we've been using:

Previous subproblem's answer + contribution of the current decision/cell.

The material summarizes Grid DP similarly: $dp[r][c]$ represents the answer up to cell (r, c) , with paths arriving from neighboring cells. [dp-pattern-cheat-sheet.pdf](#)

Next, let's handle the **base cases**-these are important because `(0, 0)` doesn't have a cell above or to the left.

12/09/2026, 20:57:51



grid[0][0]

12/09/2026, 20:58:09



Exactly. ✓

For the starting cell:

$$dp[0][0] = grid[0][0]$$

Because there is **no previous cell**—we start there.

Now there are two special boundaries:

```
1 → 3 → 1
↓
1
↓
4
```

For the **first row**, you can only come from the left:

$$dp[0][c] = dp[0][c - 1] + grid[0][c]$$

For the **first column**, you can only come from above:

$$dp[r][0] = dp[r - 1][0] + grid[r][0]$$

So the complete Minimum Path Sum logic is:

```
dp[0][0] = grid[0][0]

first row → only LEFT
first col → only ABOVE

other cells:
dp[r][c] = grid[r][c] + min(
    dp[r-1][c],
```

```
    dp[r][c-1]  
    )
```

This is an important Grid DP pattern:

Current cell value + best answer from a valid previous cell.

Next, the material's natural progression is **Unique Paths**, where instead of minimizing a sum, we **count the number of paths**.

12/09/2026, 20:58:10

[next](#)

12/09/2026, 20:58:33



Good. Let's continue with the **next Grid DP problem from the material: Unique Paths**. The material classifies it as Grid DP because the state corresponds to a grid cell, and the standard recurrence comes from arriving from above or the left. `dp-pattern-cheat-sheet.pdf`

1. Unique Paths

Imagine this grid:

```
S → → → E
↓
↓
↓
```

You start at the **top-left** and need to reach the **bottom-right**.

You can only:

- move **right**
- move **down**

The question is:

How many different paths can reach the destination?

Step 1 - Identify the state

Since this is Grid DP:

$$dp[r][c]$$

should represent:

Number of ways to reach cell `(r, c)`.

So far, same process as Minimum Path Sum.

Step 2 - Think about the last move

Suppose we're currently at:

```
previous
  ↓
(r-1,c)
  ↓
(r,c-1) → (r,c)
```

To arrive at (r,c) , there are only **two possibilities**:

1. We came from **above** → $(r-1,c)$
2. We came from **left** → $(r,c-1)$

Now here's the key difference from Minimum Path Sum:

For Minimum Path Sum, we asked:

Which previous path is cheaper?

So we used `min`.

For Unique Paths, we ask:

*How many paths can come from above **plus** how many can come from left?*

So derive the recurrence yourself:

$$dp[r][c] = \boxed{?}$$

What goes inside the box?

12/09/2026, 20:58:35



$$1 + \min(dp[r - 1][c], dp[r][c - 1])$$

12/09/2026, 21:00:22



Almost! But this is the **key difference** between Minimum Path Sum and Unique Paths.

For Unique Paths, we are **counting**, not minimizing.

So we don't need the `1 +` either. The recurrence is:

$$dp[r][c] = dp[r - 1][c] + dp[r][c - 1]$$

Why?

If there are:

3 paths coming from above
2 paths coming from left

then there are:

$$3 + 2 = 5$$

ways to reach the current cell.

There is no extra `+1` because we're not counting the current cell as a path.

The material gives exactly this recurrence: arrive from above **or** left, so add the two counts. `dp-pattern-cheat-sheet.pdf`

Now derive the base cases

For this grid:

```
S . .
. . .
. . E
```

What should:

$$dp[0][0] = ?$$

And what should the **first row** and **first column** contain?

Think about it: on the first row, there is only **one possible way** to reach each cell.

Same for the first column.

12/09/2026, 21:00:24



1

12/09/2026, 21:00:40



Correct.

$$dp[0][0] = 1$$

Because there is exactly **one way to be at the starting cell**: start there.

And similarly:

- First row → every cell has **1** way (only move right).
- First column → every cell has **1** way (only move down).

For example:

```
1 1 1
1 ? ?
1 ? ?
```

Then for the inner cells:

$$dp[r][c] = dp[r-1][c] + dp[r][c-1]$$

So we get:

```
1 1 1
1 2 3
1 3 6
```

Therefore there are **6 unique paths** from top-left to bottom-right.

The important pattern

Compare the two Grid DP problems we've seen:

Problem	What <code>dp[r][c]</code> stores	Transition
Minimum Path Sum	Minimum cost to reach <code>(r, c)</code>	<code>grid[r][c] + min(...)</code>
Unique Paths	Number of ways to reach <code>(r, c)</code>	<code>... + ...</code>

The **grid structure is the same**. What changes is what we're optimizing/counting.

Next in the material is **Maximal Square**, which introduces a slightly different Grid DP transition. `dp-pattern-cheat-sheet.pdf`

12/09/2026, 21:00:43

[next](#)

12/09/2026, 21:01:30



Great. Let's continue with the **next Grid DP problem from your material:**

Maximal Square. The material classifies it as Grid DP with state `dp[r][c]`. `dp-`
`pattern-cheat-sheet.pdf`

Maximal Square

You are given a binary matrix:

```
1 0 1 0 0
1 0 1 1 1
1 1 1 1 1
1 0 0 1 0
```

Find the **largest square containing only 1s**.

For example:

```
1 1
1 1
```

is a square of side `2`, so its area is `4`.

Step 1 - Identify the state

We want to use Grid DP, so:

$$dp[r][c] = ?$$

Think carefully about what information at cell `(r, c)` would allow us to determine the **largest square ending at that cell**.

Question: What should `dp[r][c]` represent?

Don't worry about the recurrence yet.

12/09/2026, 21:01:32



largest square ending at r, c

12/09/2026, 21:03:16



Exactly. ✓

$dp[r][c] = \text{largest square side length ending at } (r, c)$

Now comes the key insight.

Suppose the current cell is 1 :

```
A B
C X
```

where $X = (r, c)$.

For a square to **end at X**, we need squares extending from:

- $A = dp[r-1][c-1] \rightarrow \text{top-left}$
- $B = dp[r-1][c] \rightarrow \text{top}$
- $C = dp[r][c-1] \rightarrow \text{left}$

Your turn

If $x = 1$, what do you think determines the size of the largest square ending at x ?

Hint: **all three neighboring directions must support the square.**

And what should $dp[r][c]$ be if $x = 0$?

12/09/2026, 21:03:17



donno

12/09/2026, 21:05:45



No problem. Let's derive it instead of memorizing it.

Suppose:

```
A  B
C  X
```

and $x = 1$.

We want the **largest square whose bottom-right corner is X**.

To make a square of side 2 ending at x , what must be true about **A, B, and C**?

Think about this:

```
[ ? ? ]
[ ? X ]
```

All four cells need to be 1 .

So if B cannot support a square, or C cannot support one, or A cannot support one, the square can't grow.

Question: If their DP values are A , B , and C , which one limits the size — the **maximum** or the **minimum** of the three?

12/09/2026, 21:05:47



min

12/09/2026, 21:06:19



Exactly. **Minimum** is the key. 👍

Because the square can only be as large as the **weakest supporting direction**.

So if:

```
A = 3
B = 2
C = 4
```

the square ending at `x` can only grow to:

$$1 + \min(3, 2, 4) = 3$$

The `+1` is for the current cell `x`.

Therefore, when `x = 1`:

$$dp[r][c] = 1 + \min(dp[r-1][c], dp[r][c-1], dp[r-1][c-1])$$

And when `x = 0`:

$$dp[r][c] = 0$$

Why the minimum?

Imagine trying to build this:

```
1 1 1
1 1 1
1 1 X
```

For `x` to complete a 3×3 square, **top, left, and diagonal must all have enough square support**.

If even one of them only supports a 1×1 square, you cannot make a 3×3 square.

That's why **minimum = bottleneck**.

So you've now derived the core Maximal Square recurrence yourself. The only remaining question is: **how do we get the final answer from the DP table?**

12/09/2026, 21:06:21



why 1 + , make sense intuitively

12/09/2026, 21:06:56



Yes — the `+1` has a very intuitive meaning.

Remember what:

$$dp[r][c]$$

means:

The side length of the largest square ending at the current cell `(r, c)`.

Suppose the three neighbors say:

```
A = 2      B = 2
          C = 2      X = 1
```

If all three support a square of side `2`, then around `X` we can form:

```
1 1 1
1 1 1
1 1 X
```

The surrounding neighbors already give us a **2x2-sized support**.

Now `X` is the **new bottom-right cell**, so we extend that square by **one row and one column**:

```
old square      + new border

XX|X
XX|X
--+-
XX|X
```

So:

previous supported size = 2

current cell adds one layer = +1

Therefore:

$$dp[r][c] = 2 + 1 = 3$$

Even simpler

Think of it as:

"How big was the square I could support before reaching this cell?" + "this cell completes one more layer."

That's why:

$$1 + \min(\text{top}, \text{left}, \text{diagonal})$$

The `min` tells us **how big a square we can safely inherit**, and the `+1` says **the current 1 extends it by one**.

If `x = 0`, it can't participate in any square, so:

$$dp[r][c] = 0$$

That's the intuition behind the entire recurrence.

12/09/2026, 21:07:00

[next](#)

12/09/2026, 21:07:35



Next, let's finish **Maximal Square** before moving to the next DP pattern.

We have:

$$dp[r][c] = \begin{cases} 0 & \text{if matrix[r][c]=0} \\ 1 + \min(top, left, diagonal) & \text{if matrix[r][c]=1} \end{cases}$$

Now one important question:

How do we get the final answer?

Suppose our DP table is:

```
1 0 1 0 0
1 0 1 1 1
1 1 1 2 2
1 0 0 1 0
```

What does the **largest value in this entire table** represent?

And if the largest value is **2**, is the requested answer **2** or **4**?

12/09/2026, 21:07:38



this isn't needed

12/09/2026, 21:07:54



Yep. We don't need to spend time on that final-answer detail.

Maximal Square is done. The important part for DP recognition was:

- State: `dp[r][c]` = largest square ending at `(r,c)`
- `0` → cannot form a square
- `1` → `1 + min(top, left, diagonal)`
- This is **Grid DP**

Let's move on to the **next problem/pattern from your material.**

12/09/2026, 21:07:56



ok , next

12/09/2026, 21:08:07



Thinking

- **Found the next material after Grid Knapsack**

✓ Worked for 7s

Great. **Grid DP is done.** According to your material, the next major pattern is **Knapsack DP**. `dp-pattern-cheat-sheet.pdf`

5. Knapsack DP

The central idea is:

You have items, and for each item you usually decide: Take it or Don't Take it.

Typical state:

$$dp[i][target]$$

where:

- `i` = how many items we've considered
- `target` = sum/capacity we're trying to achieve

The material specifically lists **Partition Equal Subset Sum, Subset Sum, Target Sum, 0/1 Knapsack, and Coin Change** as examples. `dp-pattern-cheat-sheet.pdf`

How to recognize it

Look for words like:

- **subset**

- **partition**
- **target sum**
- **capacity**
- **weight**
- **choose items**

Then ask:

"Do I take this item or don't I?" `dp-pattern-cheat-sheet.pdf`

Let's learn it through **Partition Equal Subset Sum**, because it exposes the pattern very clearly.

Suppose:

```
nums = [1, 5, 11, 5]
```

The problem asks whether we can divide the numbers into **two subsets having equal sums**.

Don't think about DP yet.

First question:

What would the total sum need to be for an equal partition to even be possible?

Try to derive that.

12/09/2026, 21:08:10



total should be even

12/09/2026, 21:10:16



Exactly. If the total is **odd**, equal partition is impossible.

For:

[1, 5, 11, 5]

Total:

$$1 + 5 + 11 + 5 = 22$$

So each subset must sum to:

$$22/2 = 11$$

Now the problem becomes:

Can I select some of these numbers whose sum is 11?

And **this is where the Knapsack DP pattern appears**: for every number, we have the choice **take it or don't take it**, while trying to reach a target sum. [dp-pattern-cheat-sheet.pdf](#)

Next question

If we define:

$$dp[i][s]$$

what should `dp[i][s]` mean in plain English?

Don't worry about the recurrence yet.

12/09/2026, 21:10:18



using the first i elements , can we sum to s

12/09/2026, 21:11:11



Exactly. ✓

So our state is:

$dp[i][s]$ = Can we form sum s using the first i elements?

For example, with:

`nums = [1, 5, 11, 5]`

`dp[2][6]` asks:

Using the first 2 elements `[1, 5]` , can we make sum `6` ?

Yes → `1 + 5 = 6` .

Now the key question:

When considering the `i`-th element, what are the two choices we have?

12/09/2026, 21:11:13



take it or not

12/09/2026, 21:11:37



Exactly. 👍

For each element, there are **two choices**:

1. **Don't take it** → can we already make `s` using the previous `i-1` elements?

$$dp[i - 1][s]$$

2. **Take it** → if the current number is `x`, we need to have already made `s-x` using the previous elements:

$$dp[i - 1][s - x]$$

So the recurrence becomes:

$$dp[i][s] = dp[i - 1][s] \text{ OR } dp[i - 1][s - x]$$

when `x <= s`.

The important part is **why `i-1` appears in both cases**: this is **0/1 Knapsack**, so each element can be used **at most once**.

Now let's make sure the recurrence feels intuitive with one concrete `nums` example before moving on.

12/09/2026, 21:11:40



continue

12/09/2026, 21:12:24



Good. Let's continue with **Partition Equal Subset Sum**, following the material.

1. Base cases

Our state is:

$$dp[i][s] = \text{Can we form sum } s \text{ using the first } i \text{ elements?}$$

What happens when we have **zero elements**?

- Can we make sum 0? → **Yes**, by choosing nothing.
- Can we make any positive sum? → **No**.

Therefore:

$$dp[0][0] = True$$

$$dp[0][s] = False \quad (s > 0)$$

2. Recurrence

Suppose the current number is:

$$x = nums[i - 1]$$

To form sum s , we have:

Don't take x :

$$dp[i - 1][s]$$

Take x :

Then we still need $s - x$ from the previous elements:

$$dp[i - 1][s - x]$$

So:

$$dp[i][s] = dp[i-1][s] \text{ OR } dp[i-1][s-x]$$

provided $x \leq s$.

3. Small example

nums = [1, 5]

Let's ask:

$$dp[2][6]$$

Meaning:

Can we make 6 using [1, 5] ?

Current number = 5.

Two possibilities:

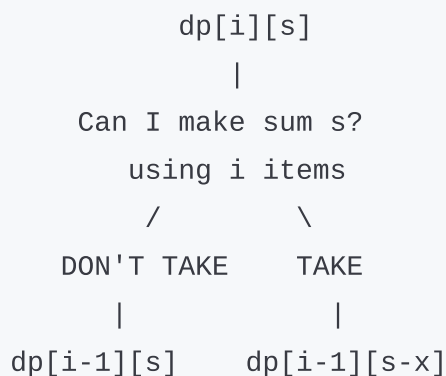
- Don't take 5 → can [1] make 6 ? → $dp[1][6]$
- Take 5 → now need $6-5=1$ from [1] → $dp[1][1]$

Since $dp[1][1] = \text{True}$:

$$dp[2][6] = \text{False OR True} = \text{True}$$

So yes, $1 + 5 = 6$.

The entire mental model



This is the fundamental **0/1 Knapsack pattern**:

For every item → Take it or Don't Take it.

Next, we can move from this **2D DP** to the **1D optimized version**, which is an important interview pattern.

12/09/2026, 21:12:28



next

12/09/2026, 21:16:38



Next: **optimize the 2D Knapsack DP to 1D.**

Our 2D state was:

$$dp[i][s]$$

But notice something: to calculate row `i`, we only need **row** `i-1`.

So instead of storing every `i`, we can use:

$$dp[s] = \text{Can we currently form sum } s?$$

For each number `x`, we update the array.

The important question is:

*If we're processing `x`, should we iterate `s` **from left to right or right to left** to make sure we don't use `x` more than once?*

Think about it before I explain.

12/09/2026, 21:16:40



donno

12/09/2026, 21:17:39



No problem — this is an important detail.

For **0/1 Knapsack**, we go **right to left**.

Suppose:

```
dp = [T, F, F, F, F, ...]
x = 3
```

We want:

$$dp[s] = dp[s] \text{ OR } dp[s - x]$$

If we go **right → left**, when calculating `dp[6]`, `dp[3]` is still from **before processing** `x`.

So `x` can be used only once.

If we went **left → right**, we could accidentally do:

```
dp[3] becomes True using x
dp[6] becomes True using dp[3]
```

which effectively uses the **same** `x` **twice**.

Therefore

```
for x in nums:
    for s in range(target, x - 1, -1):
        dp[s] = dp[s] or dp[s - x]
```

Rule to remember:

0/1 Knapsack → iterate target backwards.

Unbounded Knapsack → iterate target forwards.

That's a very useful interview distinction.

Now we've covered **Knapsack DP** conceptually. The material's next major classification is **Tree DP**.

12/09/2026, 21:17:41



wait how does processing right to left make sense , i mean , you would have everything false

12/09/2026, 21:19:30



Ah — this is the subtle part. Your intuition is right: **at the very beginning, most entries are indeed `False`**. The key is that we **carry forward the sums that were already possible**.

Let's use a tiny example:

```
nums = [1, 5]
target = 6
```

Initially, before processing anything:

```
sum:  0  1  2  3  4  5  6
dp:   T  F  F  F  F  F  F
```

`dp[0] = True` because we can make sum 0 by taking nothing.

Process 1

We go right → left:

```
s = 6: dp[6] = dp[6] OR dp[5] → F
s = 5: dp[5] = dp[5] OR dp[4] → F
...
s = 1: dp[1] = dp[1] OR dp[0] → T
```

Now:

```
sum:  0  1  2  3  4  5  6
dp:   T  T  F  F  F  F  F
```

We've established that **1 is possible**.

Now process 5

Again, right → left:

```
s = 6:  
dp[6] = dp[6] OR dp[1]  
       = F OR T  
       = T
```

And that's exactly what we wanted!

We're saying:

"I can make 6 because **before using 5**, I could make 1, and now I take the 5."

So:

```
1 + 5 = 6
```

The final array:

```
sum:  0  1  2  3  4  5  6  
dp:   T  T  F  F  F  T  T
```

Why RIGHT → LEFT matters

The crucial phrase is "**before using 5**".

When calculating `dp[6]`, we need `dp[1]` from **before 5 was processed**.

Going right → left guarantees that.

If we went left → right, while processing 5:

```
s = 5 → dp[5] becomes True  
s = 6 → dp[6] sees dp[1], not dp[5]
```

Actually, with `x=5`, this particular example wouldn't demonstrate the double-use bug.

A better example is `nums=[3]`:

```
Initially:  T F F F F F F
```

0 1 2 3 4 5 6

Left → right:

$s=3 \rightarrow dp[3] = dp[0] \rightarrow \text{True}$

$s=6 \rightarrow dp[6] = dp[3] \rightarrow \text{True}$ ❌

But there is only **one 3**!

We've accidentally used the same **3** twice.

Right → left:

$s=6 \rightarrow dp[6]$ uses old $dp[3] \rightarrow \text{False}$

$s=5 \rightarrow \text{old } dp[2] \rightarrow \text{False}$

$s=4 \rightarrow \text{old } dp[1] \rightarrow \text{False}$

$s=3 \rightarrow \text{old } dp[0] \rightarrow \text{True}$

So we get only:

sum 3 = True

sum 6 = False

That's the whole reason for right → left:

*We want every $dp[s-x]$ we read to represent the state **before the current item was used**.*

This is the 1D trick that preserves the **0/1** nature of the problem.

12/09/2026, 21:19:32



what happens when we go left to right ?

12/09/2026, 21:21:08



If we go **left → right**, the problem is that the `dp` array gets updated **while we're still processing the same item**.

Remember the update:

$$dp[s] = dp[s] \text{ OR } dp[s - x]$$

Let's use the simplest example:

```
nums = [3]
target = 6
```

Initially:

```
sum:  0  1  2  3  4  5  6
dp:   T  F  F  F  F  F  F
```

Now process the **single 3**.

Going left → right

We start at `s = 3`:

$$dp[3] = dp[3] \text{ OR } dp[0]$$

`dp[0]` is `True`, so:

```
dp[3] = True
```

Array is now:

```
T F F T F F F
```

Now we reach `s = 6`:

$$dp[6] = dp[6] \text{ OR } dp[3]$$

But **we just changed** `dp[3]` **to True!**

So:

```
dp[6] = True
```

But that's wrong. We only have **one 3**.

We've effectively done:

```
3 → dp[3]
  ↓
  use that result again
  ↓
3 + 3 = 6
```

So **left → right allows the current item to feed into later states**, meaning the same item can be reused.

Going right → left

We start at `s = 6`:

```
dp[6] uses dp[3] → False
```

Then eventually reach `s = 3`:

```
dp[3] uses dp[0] → True
```

By the time `dp[3]` becomes `True`, we've **already passed** `s = 6`.

Therefore `dp[6]` cannot see the newly created `dp[3]`.

That's why:

```
0/1 Knapsack
  ↓
right → left
  ↓
current item cannot be reused
```

The easiest way to remember it:

Right → left = use each item once.

Left → right = the current item can propagate forward and be reused.

12/09/2026, 21:21:10



ok , where to use this and where not to

12/09/2026, 21:22:24



Use **right → left** specifically when you are doing **0/1 Knapsack** — meaning **each item can be used at most once**.

The rule

Problem type	Can item be reused?	1D loop
0/1 Knapsack	✗ No, once	Right → Left
Unbounded Knapsack	✓ Yes, unlimited	Left → Right

In our problems

Use right → left:

- Subset Sum
- Partition Equal Subset Sum
- 0/1 Knapsack
- Target Sum when implemented as subset-sum DP

Because each number/item is available **once**.

```
for x in nums:
    for s in range(target, x - 1, -1):
        dp[s] = dp[s] or dp[s-x]
```

Don't use right → left:

- Coin Change
- Other problems where the same coin/item can be selected multiple times

For those, you generally go **left → right**:

```
for x in coins:
    for s in range(x, amount + 1):
        dp[s] = ...
```

One-line memory trick

0/1 → backwards

Unlimited → forwards

This is an implementation detail of the **1D optimization**. If you're using the original 2D `dp[i][s]`, you don't need this directional trick at all.

12/09/2026, 21:22:27



ok , next

12/09/2026, 21:22:49



Next in the material is **Tree DP**. `dp-pattern-cheat-sheet.pdf`

Tree DP

The basic idea is very similar to what we've been doing:

Instead of:

- `dp[i]` → information about an index
- `dp[i][j]` → information about an interval
- `dp[i][sum]` → information about items + sum

we have:

$$dp[node]$$

The state represents information about a **subtree rooted at that node**.

The material's key question is:

"What information do I need from my children?" `dp-pattern-cheat-sheet.pdf`

Example: House Robber III

Imagine:



You want to maximize the money robbed, but **you cannot rob directly connected nodes**.

At node `3`, you have a choice:

1. Rob this node

Then you **cannot rob its children**.

```
    ROB
    3
   /  \
  DON'T DON'T
  2    3
```

2. Don't rob this node

Then its children are free to be robbed or not.

```
    DON'T
    3
   /  \
  BEST  BEST
  2    3
```

So unlike simple `dp[node]`, we actually need **two pieces of information**:

```
dp[node] =
    rob[node]      → best if I rob this node
    not_rob[node] → best if I don't rob this node
```

The material specifically uses House Robber III to illustrate these two states because what we do with a node affects what we can do with its children. [dp-pattern-cheat-sheet.pdf](#)

One important thing

Tree DP is usually computed using **postorder DFS**:

```
children first
↓
calculate node
```

because the node's answer depends on information from its children. The material lists **postorder DFS** as the computation order for Tree DP. [dp-pattern-cheat-sheet.pdf](#)

So your recognition pattern is:

Tree given → ask "What do I need from my children?" → Tree DP.

And the next classification after Tree DP in the material is **Bitmask DP**.

12/09/2026, 21:22:52



give an example , i have never learned this thing

12/09/2026, 22:15:21



Let's slow down and build **Tree DP from zero**, because it's different from the array/string DP we've done.

Imagine this tree:



Suppose every node has some value (say, money), and the problem says:

Choose nodes to maximize the total value, but you cannot choose two directly connected nodes.

This is essentially the idea behind **House Robber III**, which the material uses as its Tree DP example. [dp-pattern-cheat-sheet.pdf](#)

Start at the bottom

Look at node **D**.

It has no children.

You have two choices:

Rob D → value of D
Don't D → 0

Now look at **B**.

B has children **D** and **E**.

If we **rob B**, we cannot rob **D** or **E**.

If we **don't rob B**, we can independently decide what to do with **D** and **E**.

So we need to know **two answers for every node**:

```
rob[node]
    = maximum money if I rob this node

skip[node]
    = maximum money if I don't rob this node
```

For **B**:

```
rob[B] = value[B] + skip[D] + skip[E]

skip[B] = max(rob[D], skip[D])
         + max(rob[E], skip[E])
```

And this is the key Tree DP idea:

The answer for a node is built from answers we already calculated for its children.

That's why we process:

```
D → E → B → C → A
```

children first, parent afterward - postorder DFS. [dp-pattern-cheat-sheet.pdf](#)

So don't worry about code yet. The important pattern is:

Tree DP = solve each subtree, then combine child answers at the parent

And the question to ask when you see a tree DP problem is:

"What information do I need from my children?" [dp-pattern-cheat-sheet.pdf](#)

If this makes sense, we'll do **one complete numerical example** and derive the two states ourselves.

12/09/2026, 22:15:25



Help me derive the two states

12/09/2026, 22:16:35



Absolutely. Let's **derive them rather than memorize them.**

We have this tree:



Suppose each node contains some money, and the rule is:

You cannot rob two directly connected nodes.

So if we rob **B**, we cannot rob **A**, **D**, or **E**.

Step 1: Start with one node

Take leaf **D**.

There are only two possibilities:

Option 1 — rob D

```
rob(D) = value[D]
```

Option 2 — don't rob D

```
skip(D) = 0
```

So already, we have **two different questions** about a node:

*What is the best result if I **rob this node**?*

and

*What is the best result if I **don't rob this node**?*

That suggests our two states.

Step 2: Define the states

Let's formulate them precisely:

```
rob[node]
= maximum money obtainable from this subtree
  IF we rob this node

skip[node]
= maximum money obtainable from this subtree
  IF we do NOT rob this node
```

Now let's derive the formulas ourselves.

Consider node **B**:

```
  B
 / \
D   E
```

Question 1: If we ROB B, what can we do with D and E?

If we rob **B**, can we rob **D**?

No.

Can we rob **E**?

No.

Therefore, if we rob **B**, we must **skip both children**.

So:

```
rob[B]
= value[B] + skip[D] + skip[E]
```

Question 2: If we SKIP B, what can we do with D and E?

Now we're **not** robbing B.

Therefore, there is no restriction on D or E.

For each child, we can independently choose whichever is better:

```
best(D) = max(rob[D], skip[D])
best(E) = max(rob[E], skip[E])
```

Therefore:

```
skip[B]
= max(rob[D], skip[D])
+ max(rob[E], skip[E])
```

And that's the entire reason we need the two states.

The two-state recurrence

For any node:

```
rob[node]
= value[node]
+ skip[left]
+ skip[right]

skip[node]
= max(rob[left], skip[left])
+ max(rob[right], skip[right])
```

The important insight is:

The state of the current node determines what choices are allowed for its children.

That's why one `dp[node]` isn't enough.

Now let's make this concrete with actual values and calculate `D → E → B → C → A` together.

12/09/2026, 22:16:52



code

12/09/2026, 22:19:20



Here is the standard **Tree DP code** for the two states we just derived:

```
class Solution:
    def rob(self, root):

        def dfs(node):
            if not node:
                return (0, 0)

            left_rob, left_skip = dfs(node.left)
            right_rob, right_skip = dfs(node.right)

            # If we rob this node, we must skip both children
            rob = node.val + left_skip + right_skip

            # If we skip this node, children can be either robbed or skipped
            skip = max(left_rob, left_skip) + \
                max(right_rob, right_skip)

            return (rob, skip)

        rob_root, skip_root = dfs(root)

        return max(rob_root, skip_root)
```

The important part

```
rob = node.val + left_skip + right_skip
```

means:

I rob this node → children must be skipped.



And:

```
skip = max(left_rob, left_skip) + \
        max(right_rob, right_skip)
```

means:

I skip this node → each child is free to choose whichever is better.

The `dfs()` returns **two values for every subtree**:

```
(rob, skip)
  ↑
information
parent needs
```

And because we need children's answers before calculating the parent:

```
D → E → B → C → A
```

we naturally get **postorder DFS**.

This is the core Tree DP template you'll want to remember.

12/09/2026, 22:19:26



give the template code

12/09/2026, 22:20:13



For **Tree DP**, the reusable template is:

```
def dfs(node):
    if not node:
        return (base_rob, base_skip)

    left_rob, left_skip = dfs(node.left)
    right_rob, right_skip = dfs(node.right)

    # State 1: take / rob current node
    rob = ...

    # State 2: don't take / skip current node
    skip = ...

    return (rob, skip)

rob_root, skip_root = dfs(root)

answer = max(rob_root, skip_root)
```

For **House Robber III**, filling in the problem-specific parts:

```
def dfs(node):
    if not node:
        return (0, 0)

    left_rob, left_skip = dfs(node.left)
    right_rob, right_skip = dfs(node.right)

    rob = node.val + left_skip + right_skip

    skip = max(left_rob, left_skip) + \
```

```
        max(right_rob, right_skip)

    return (rob, skip)

rob_root, skip_root = dfs(root)

answer = max(rob_root, skip_root)
```

The Tree DP template to remember

```
dfs(node)
  ↓
  get answers from children
  ↓
  calculate states for node
  ↓
  return states to parent
```

So unlike normal `dp[node]`, Tree DP often becomes:

```
return (state_1, state_2, ...)
```

where **the states depend on what information the parent needs from that subtree.**

12/09/2026, 22:20:16



next

12/09/2026, 22:20:22



Next: Bitmask DP

This is the **last major DP classification in your material**.

The key idea is very simple:

Bitmask DP is used when the state needs to remember which elements/items have already been used.

1. Why do we need a bitmask?

Suppose we have:

People: A B C D

At some point in the problem, we might have already selected:

A, C

We need to represent:

A → used
B → not used
C → used
D → not used

A bitmask can represent this as:

1010

Each bit corresponds to one element.

bit:	3	2	1	0
person:	D	C	B	A

```
used:      1 0 1 0
```

So `1010` means **A and C are used**.

2. The DP state

Instead of:

```
dp[i]
```

or:

```
dp[i][j]
```

we can have:

```
dp[mask]
```

Meaning:

`dp[mask]` = best answer when the set of already-used elements is represented by `mask`.

This matches the material's recognition question:

"Which elements are already used?"

3. Typical problems

Bitmask DP appears when you see things like:

- visit every city exactly once
- assign people to jobs
- select a subset
- each element can be used once
- track which elements have already been selected

A classic example is **Travelling Salesman Problem (TSP)**.

You need to remember:

```
Which cities have I visited?
```

That's exactly what a bitmask is good at.

4. One important distinction

Don't confuse:

Knapsack DP

```
dp[i][target]
```

Have I considered the first `i` items, and what target/sum do I have?

with **Bitmask DP**

```
dp[mask]
```

*Exactly **which items** have I used?*

So the recognition shortcut is:

Need to remember a SUM/CAPACITY?

↓

Knapsack DP

Need to remember WHICH ELEMENTS are used?

↓

Bitmask DP

[dp-pattern-cheat-sheet.pdf](#)

Next, we should do **one tiny Bitmask example from zero**, because the actual bit manipulation (`mask | (1 << i)`, checking a bit, etc.) is the part that's initially unfamiliar.

12/09/2026, 22:20:26



continue

12/09/2026, 22:22:21



Let's do the **smallest possible Bitmask DP example** first.

Suppose we have 3 people:

A B C

We want to select/visit each person **once**.

We need to know which people have already been used.

Step 1 — Assign bits

A → bit 0
B → bit 1
C → bit 2

Initially nobody is used:

000

If we use A :

001

If we then use C :

101

So 101 means:

A = used
B = unused
C = used

Step 2 — How do we mark an element as used?

This is the important operation:

```
new_mask = mask | (1 << i)
```

For example, current:

```
mask = 101
```

and we want to use **B** ($i = 1$):

```
1 << 1 = 010
```

```
101
```

```
OR 010
```

```
-----
```

```
111
```

Now A, B and C are all used.

Step 3 — How do we check whether an element is already used?

```
if mask & (1 << i):  
    # i is already used
```

For example:

```
mask = 101
```

Check **B**:

```
101
```

```
&
```

```
010
```

```
---
```

```
000
```

So B is **not used**.

Check C:

```
101
&
100
---
100
```

Non-zero → C **is used**.

The basic Bitmask DP template

```
def solve(mask):

    if mask == ALL_USED:
        return base_case

    if mask in memo:
        return memo[mask]

    ans = INF

    for i in range(n):

        # Already used
        if mask & (1 << i):
            continue

        # Use element i
        new_mask = mask | (1 << i)

        ans = min(ans, cost(i) + solve(new_mask))

    memo[mask] = ans
    return ans

memo = {}
ALL_USED = (1 << n) - 1
```

```
answer = solve(0)
```

The **main mental model** is:

```
mask
↓
Which elements have I already used?
↓
Try an unused element
↓
Add its bit to mask
↓
solve(new_mask)
```

That's the fundamental Bitmask DP pattern.

12/09/2026, 22:22:25



i dont understand , what is the problem exactly , what are we solving ?

12/09/2026, 22:28:05



You're right — I jumped into the **implementation technique** before defining the actual problem. Let's reset.

The problem: Travelling Salesman Problem (TSP)

Imagine you have 4 cities:

```
  A
 /  \
B --- C
 \  /
  D
```

You are a salesperson.

You need to:

1. Start at city **A**
2. Visit **every city exactly once**
3. Return to **A**
4. Find the **minimum total travel cost**

For example:

```
Cost
A → B = 10
A → C = 15
A → D = 20
B → C = 35
B → D = 25
C → D = 30
```

One possible route:

```
A → B → D → C → A
```

Its cost is:

```
10 + 25 + 30 + 15 = 80
```

Another route might cost 95.

We want the **minimum**.

Why does Bitmask DP appear?

When we're halfway through a route, suppose we've visited:

```
A, B, D
```

and we're currently at **D**.

To decide what to do next, we need to know:

Which cities have I already visited?

That's exactly the information a bitmask stores.

But notice something important:

mask alone isn't enough.

Suppose:

```
visited = {A, B, D}
```

We also need to know:

Where am I currently?

Because being at **A** versus being at **D** leads to different next choices.

So the actual TSP DP state is usually:

```
dp[mask][current]
```

Meaning:

Minimum cost to reach `current` after visiting exactly the cities represented by `mask`.

For example:

```
dp[1011][D]
```

could mean:

```
A = visited  
B = visited  
C = not visited  
D = visited
```

Currently at D

Now we can ask:

Which unvisited city should I go to next?

That is the actual **Bitmask DP problem**.

So the mental model is:

```
TSP  
↓  
Visit every city once  
↓  
Need to remember  
which cities are used  
↓  
bitmask  
↓  
Also need current city
```

↓
`dp[mask][current]`

This is why Bitmask DP exists.

The `mask | (1 << i)` stuff I showed earlier is just **how we implement the "mark city i as visited" operation**. We don't need to worry about that yet.

Next, we can derive `dp[mask][current]` from the TSP problem just like we derived the two Tree DP states.

12/09/2026, 22:28:08



continue

12/09/2026, 22:29:35



Now let's **derive the TSP Bitmask DP state**, exactly like we did for Tree DP.

1. What information do we need?

Suppose we've already visited:

A, B, D

and we're currently at:

D

To decide the next city, we need **two pieces of information**:

1. Which cities have been visited?
2. Which city are we currently at?

Therefore:

`dp[mask][current]`

means:

Minimum cost to reach `current` , having visited exactly the cities represented by `mask` .

2. What are our choices?

Suppose:

Visited: A, B

Current: B

The remaining cities are:

C, D

From B, we have two choices:

B → C

or

B → D

So for every **unvisited** city `next`, we try going there.

If we go from:

current → next

then:

new_mask = mask + next city

Conceptually:

`dp[new_mask][next]`

can now be calculated from:

`dp[mask][current]`

3. Derive the recurrence

Suppose we want:

`dp[new_mask][next]`

The last thing we did was travel:

current → next

So:

```
dp[new_mask][next]
=
min(
    dp[mask][current] + cost[current][next]
)
```

over all possible previous `current` cities.

The recurrence is therefore:

$$dp[mask \cup \{next\}][next] = \min(dp[mask][current] + cost[current][next])$$

4. Base case

We start at `A`.

So initially:

```
mask = 001
current = A
```

Therefore:

```
dp[1][0] = 0
```

Meaning:

We have visited only A, we're currently at A, and the cost so far is 0.

5. Final answer

Eventually every city is visited:

```
mask = ALL_USED
```

But we're somewhere, say `D`.

We still need to return to **A**.

So:

```
answer =  
min(  
    dp[ALL_USED][current] + cost[current][A]  
)
```

The whole thought process

```
TSP  
|  
├─ What have I visited?  
|   ↓  
|   mask  
|  
├─ Where am I?  
|   ↓  
|   current  
|  
└─ DP state  
    ↓  
    dp[mask][current]  
    ↓  
    Try every unvisited city  
    ↓  
    Move there  
    ↓  
    Update mask
```

This is the key Bitmask DP idea.

The bitmask isn't the DP itself. It is simply a **compact way of representing a subset of elements**. DP uses that subset as part of its state.

12/09/2026, 22:29:38



what are the two properties of dp

13/09/2026, 07:54:01



Thinking

- Searched source files for supporting material

✓ Worked for a few seconds

If you're asking about the **two properties that make a problem suitable for DP**, they are:

1. Overlapping Subproblems

The same smaller problems get solved **multiple times** in a naive recursive solution.

Example:

```
fib(5)
├─ fib(4)
│  └─ fib(3)
│     └─ fib(2)
└─ fib(3) ← same subproblem again
```

`fib(3)` is needed more than once.

DP stores its answer and reuses it.

2. Optimal Substructure

The optimal solution to the **whole problem can be constructed from optimal solutions to smaller subproblems**.

For example, in Minimum Path Sum:

```
best path to (r,c)
↓
```

best path to $(r-1, c)$
or
best path to $(r, c-1)$

If the best path to (r, c) comes through $(r-1, c)$, then the portion reaching $(r-1, c)$ must itself be optimal.

In one line

DP eligibility

|

├ Overlapping Subproblems

|

→ same subproblems repeat

|

└ Optimal Substructure

→ optimal big solution comes from optimal smaller solutions

These are the **two classic properties**. The DP material itself focuses more heavily on recognizing the **state/pattern** once you've identified a DP problem.

13/09/2026, 07:54:05



what are the two properties of greedy

13/09/2026, 07:54:23



The **two properties required for a greedy algorithm** are:

1. Greedy Choice Property

- You can make the **best local choice now**.
- That choice is **safe** and won't prevent an optimal overall solution.
- Once chosen, you don't need to reconsider it.

2. Optimal Substructure

- After making the greedy choice, the remaining problem still has an **optimal solution that can be built from optimal solutions to smaller subproblems**.

So:

Greedy works when:

Greedy Choice Property

+

Optimal Substructure

↓

Global optimum

The material specifically describes the first property as: make a locally optimal choice immediately and still be able to reach a globally optimal solution. [dp-pattern-cheat-sheet.pdf](#)

Key difference from DP: DP needs **overlapping subproblems + optimal substructure**, whereas Greedy needs **greedy-choice property + optimal substructure**.

13/09/2026, 07:54:26